

ENDLESS ENGINE

Endless Engine — Türkçe Kullanım Kılavuzu

Unity idle / incremental oyun toolseti için geliştirici kullanım kılavuzu

PAKET

``com.endlessengine.idle``
v1.1.0

MOTOR

Unity 6.3 LTS

HEDEF KİTLE

**Unity ile idle / incremental
oyun geliştiren C#
geliştiricileri**

SON GÜNCELLEME

2026-05-02

Bu kılavuz Endless Engine'i sıfırdan kurarak tam çalışan bir idle oyunu yapmanız için gereken her şeyi kapsar. İlk okumada baştan sona geçin; sonra ihtiyaç duydukça ilgili bölüme dönün.

Kılavuz Haritası

Bu PDF, kaynak Markdown içeriğindeki tüm başlıkları ve örnekleri koruyarak bölüm bazlı gezinmeye uygun hale getirilmiştir.

BÖLÜM A

Başlangıç

1-3. başlıklar

BÖLÜM B

Zorunlu Çekirdek Sistemler

4-7. başlıklar

BÖLÜM C

Gelir Sistemleri

8-10. başlıklar

BÖLÜM D

Aktif Oynanış Sistemleri

11-13. başlıklar

BÖLÜM E

İlerleme Sistemleri

14-19. başlıklar

BÖLÜM F

Savaş Sistemi

20-23. başlıklar

BÖLÜM G

İçerik Sistemleri

24-34. başlıklar

BÖLÜM H

Destek Sistemleri

35-49. başlıklar

BÖLÜM I

Araçlar ve Testler

50-51. başlıklar

BÖLÜM J

Oyun Yapım Tarifleri

52-57. başlıklar

BÖLÜM K

Sorun Giderme

58-58. başlıklar

İçindekiler

Bölüm ve başlık bağlantılarına tıklayarak PDF içinde doğrudan ilgili sayfaya geçebilirsiniz.

Bölüm A — Başlangıç

1. Kurulum ve Gereksinimler
2. Temel Mimari
3. Hızlı Başlangıç

Bölüm B — Zorunlu Çekirdek Sistemler

4. ConfigRegistry
5. SaveService
6. EconomyService
7. TickEngine

Bölüm C — Gelir Sistemleri

8. GeneratorSystem
9. PassiveIncomeService
10. OfflineTimeCalculator

Bölüm D — Aktif Oynanış Sistemleri

11. ClickYieldService
12. ClickLoopService
13. HarvestLoopService

Bölüm E — İlerleme Sistemleri

14. UpgradeTreeService
15. UpgradeApplicationSystem
16. SkillTreeService
17. ResearchService
18. PrestigeStateManager
19. AscensionStateManager

Bölüm F — Savaş Sistemi

20. WaveSpawnManager
21. AutoBattleController
22. DamageSystem
23. InputProvider

Bölüm G — İçerik Sistemleri

24. BuildingService
25. PetService
26. EventService
27. ChallengeService
28. MilestoneTracker
29. QuestService
30. MinigameService
31. MergeService
32. ConversionService
33. TraitService
34. UnlockLogService

Bölüm H — Destek Sistemleri

- 35. StatisticsService
- 36. NotificationService
- 37. LeaderboardService
- 38. TutorialService
- 39. RealmSystem
- 40. AudioService
- 41. BigDouble
- 42. TimeBoostService
- 43. ZoneSystem
- 44. CursorYieldService
- 45. InventoryService
- 46. DropResolver
- 47. PlayerHealthComponent
- 48. GameFlowState ve RunSummaryData
- 49. IdleModule

Bölüm I — Araçlar ve Testler

- 50. Editor Araçları
- 51. Test Stratejisi

Bölüm J — Oyun Yapım Tarifleri

- 52. Tarif 1: Klasik Idle
- 53. Tarif 2: Aktif Clicker
- 54. Tarif 3: Hasat Loop
- 55. Tarif 4: Idle-vs
- 56. Tarif 5: Merge Idle
- 57. Tarif 6: Prestige-Heavy RPG Idle

Bölüm K — Sorun Giderme

- 58. Sorun Giderme

BÖLÜM A

Başlangıç

- 1. Kurulum ve Gereksinimler**
- 2. Temel Mimari**
- 3. Hızlı Başlangıç**

1. Kurulum ve Gereksinimler

Bağımlılıklar

Aşağıdaki Unity paketleri kurulu olmalıdır (Window → Package Manager):

Paket	Minimum Versiyon	Not
Input System	1.14.2	Kurulum sonrası Unity yeniden başlatma ister — "Yes" deyin
Addressables	2.7.6	Config yükleme için
Newtonsoft Json	3.2.2	Kayıt serileştirme için

Paket Kurulumu

1. `com.endlessengine.idle/` klasörünü projenizin `Packages/` klasörüne kopyalayın:

```

YourProject/
├── Packages/
│   └── com.endlessengine.idle/
│       ├── Runtime/
│       ├── Editor/
│       ├── Tests/
│       └── package.json

```

2. Unity editörünü açın — paket otomatik yüklenir.
3. `Packages/manifest.json` dosyasında `"testables"` dizisine `"com.unity.inputsystem"` eklendiğinden emin olun (Input System testleri için).

Hızlı Test: MinimalIdle Örneği

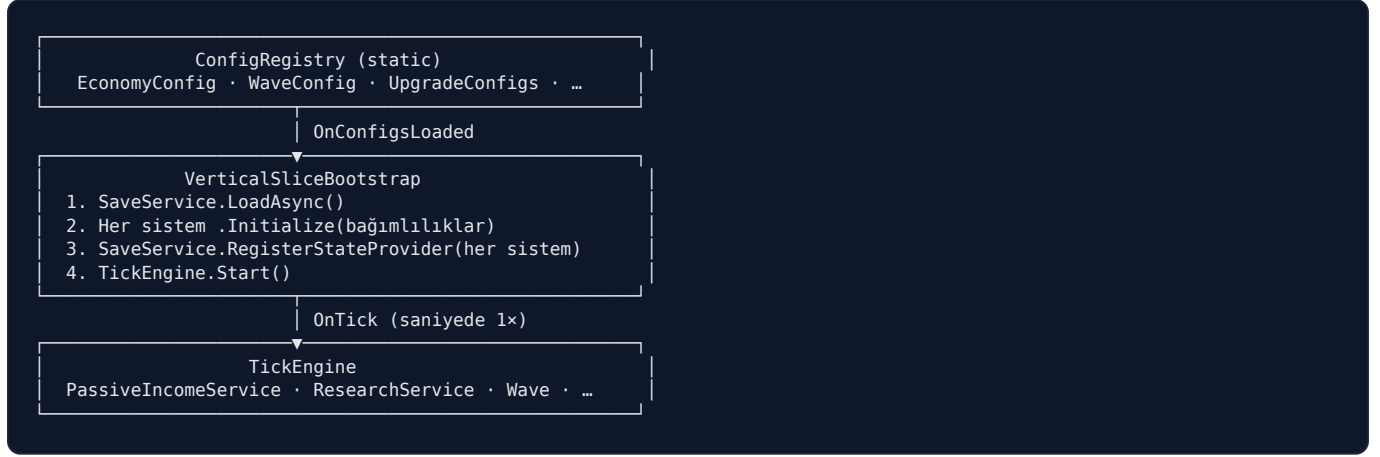
```

Window → Package Manager → Endless Engine → Samples → MinimalIdle → Import
Assets/Samples/.../MinimalIdle/Scenes/MinimalIdle.unity → Aç → Play

```

2. Temel Mimari

Genel Akış



Beş Temel Kural

Kural	Neden
Config = ScriptableObject	Kodda sihirli sayı yok; tasarımcı Inspector'dan değiştirebilir
Haberleşme = C# event	Sistemler birbirini doğrudan çağırılmaz; bağımlılık azalır
Kayıt = ISaveServiceProvider	Her sistem kendi verisini bildirir; SaveService toplar
Bağımlılık = Initialize() enjeksiyonu	new/singleton yok; birim testleri yazılabilir
Config erişimi = ConfigRegistry	Tek erişim noktası; Addressables veya test inject ile değiştirilebilir

Namespace Haritasi

EndlessEngine.Config	ConfigRegistry, tüm *ConfigSO'lar
EndlessEngine.SaveAndLoad	SaveService, ISaveStateProvider, SaveData, SaveConstants
EndlessEngine.Economy	EconomyService, CurrencyService, InventoryService, MergeService, ConversionService
EndlessEngine.Economy.Math	IBigInteger, BigInteger, DoubleNumber
EndlessEngine.Flow	TickEngine, GameFlowStateMachine
EndlessEngine.Generator	GeneratorSystem, PassiveIncomeService
EndlessEngine.Core	UpgradeApplicationSystem (static)
EndlessEngine.Upgrade	UpgradeTreeService, SkillTreeService
EndlessEngine.Research	ResearchService
EndlessEngine.Prestige	PrestigeStateManager, AscensionStateManager
EndlessEngine.Wave	WaveSpawnManager
EndlessEngine.Combat	AutoBattleController, BaseStatUpgradeProvider
EndlessEngine.Damage	DamageSystem (static)
EndlessEngine.Health	HealthSystem, PlayerHealthComponent
EndlessEngine.Input	IInputProvider, InputProviderUnity
EndlessEngine.Offline	OfflineTimeCalculator
EndlessEngine.Building	BuildingService
EndlessEngine.Pet	PetService
EndlessEngine.Events	EventService
EndlessEngine.Challenge	ChallengeService
EndlessEngine.Milestone	MilestoneTracker
EndlessEngine.Statistics	StatisticsService
EndlessEngine.Tutorial	TutorialService
EndlessEngine.Quest	QuestService
EndlessEngine.Minigame	MinigameService
EndlessEngine.Trait	TraitService
EndlessEngine.UnlockLog	UnlockLogService
EndlessEngine.Leaderboard	LeaderboardService
EndlessEngine.Notification	NotificationService
EndlessEngine.Realm	RealmConfigSystem
EndlessEngine.Audio	AudioService
EndlessEngine.Harvest	HarvestLoopService, HarvestOfflineCalculator, HarvestCursor, HarvestNode, HarvestNodeSpawner

EndlessEngine.ClickLoop	ClickLoopService, ClickLoopOfflineCalculator, ClickTarget, ClickTargetSpawner
EndlessEngine.Modules	ClickYieldService, CursorYieldService
EndlessEngine.VFX	VFXController
EndlessEngine.UI	HarvestHUDController, ClickLoopHUDController

3. Hızlı Başlangıç

Bu adımlar 15 dakikada çalışan bir idle oyun verir.

Adım 1 — Config Asset'leri Oluşturun

Tools → Endless Engine → New Game Wizard → Pure Idle → Create

Bu işlem `Assets/Configs/` altında şunları otomatik oluşturur: `EconomyConfig.asset`, `WaveConfig.asset`, `SchemaVersion.asset`, örnek `GeneratorConfig` ve `UpgradeNodeConfig` asset'leri.

Adım 2 — Sahne Hiyerarşisi

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  GeneratorSystem
  PassiveIncomeService
  UpgradeTreeService
```

Adım 3 — Bootstrap Script'i

```
using UnityEngine;
using EndlessEngine.Config;
using EndlessEngine.Economy;
using EndlessEngine.Flow;
using EndlessEngine.Generator;
using EndlessEngine.SaveAndLoad;
using EndlessEngine.Upgrade;

public class GameBootstrap : MonoBehaviour
{
    [SerializeField] private EconomyConfigSO _economyConfig;
    [SerializeField] private SaveService _saveService;
    [SerializeField] private EconomyService _economyService;
    [SerializeField] private TickEngine _tickEngine;
    [SerializeField] private GeneratorSystem _generatorSystem;
    [SerializeField] private PassiveIncomeService _passiveIncome;
    [SerializeField] private UpgradeTreeService _upgradeTree;

    private async void Awake()
    {
        // Config'leri yükle
        ConfigRegistry.InjectForTesting(economy: _economyConfig);

        // UpgradeTree'yi config'lerden başlat (Initialize() yok, HandleConfigsLoaded() var)
        _upgradeTree.HandleConfigsLoaded();

        // Economy ve diğer sistemleri başlat
        _economyService.Initialize(_upgradeTree, _saveService);
        _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService, _saveService);
        _passiveIncome.Initialize(_generatorSystem, _economyService, null);

        // Kayıt sistemi sağlayıcılarını kaydet
        _saveService.RegisterStateProvider(_economyService);
        _saveService.RegisterStateProvider(_upgradeTree);
    }
}
```

```
        _saveService.RegisterStateProvider(_generatorSystem);

        // Son olarak kayıtları yükle (sistemler kayıt sonrası güncellenir)
        await _saveService.LoadAsync();
    }
}
```

Adım 4 — UI'dan Jeneratör Satın Alma

```
public class GeneratorButton : MonoBehaviour
{
    [SerializeField] private string _generatorId = "coal-mine";
    [SerializeField] private GeneratorSystem _generators;

    public void OnClick() => _generators.TryPurchase(_generatorId);
}
```

Play tuşuna basın — altın kazanmaya başlar.

BÖLÜM B

Zorunlu Çekirdek Sistemler

4. ConfigRegistry

5. SaveService

6. EconomyService

7. TickEngine

4. ConfigRegistry

Ne yapar: Tüm `ScriptableObject` config'lerine global erişim sağlayan static merkez. Sisteme tüm ayarların tek kapıdan girdiğini garanti eder.

Erişim

```
EconomyConfigSO economy = ConfigRegistry.Economy;
WaveConfigSO wave = ConfigRegistry.Wave;
PrestigeConfigSO prestige = ConfigRegistry.Prestige;
// Diğerleri: Player, Schema, Run, Realm, UpgradeSelection, RealmRegistry
```

Prodüksiyon: Addressables ile Yükleme

Config asset'lerini Addressables grubuna ekleyin, ardından Bootstrap'te:

```
await ConfigLoadingService.LoadAllAsync(addressableLabel: "configs");
// OnConfigsLoaded event'i tetiklenir → tüm sistemler buraya abone olarak başlar
```

Geliştirme / Test: Doğrudan Enjeksiyon

```
ConfigRegistry.InjectForTesting(
    economy: _economyConfig, // null geçilen alanlar atlanır
    wave: _waveConfig,
    player: _playerConfig,
    prestige: _prestigeConfig,
    schema: _schemaVersion,
    realm: _realmIdentity,
    upgrades: _upgradeNodeArray,
    run: _runConfig,
    realmRegistry: _realmRegistry,
    upgradeSelection: _upgradeSelectionConfig
);
// Test sonrası mutlaka temizleyin:
ConfigRegistry.ClearForTesting();
```

Sadece ihtiyaç duyduğunuz alanları geçin; geri kalanları `null` kalır (erişilmezse exception fırlatır).

Kullanılabilir Property'ler

Property	Tip	Ne için
<code>ConfigRegistry.Economy</code>	<code>EconomyConfigSO</code>	Hard cap, starting gold, backend
<code>ConfigRegistry.Wave</code>	<code>WaveConfigSO</code>	Dalga sayısı, düşman scaling
<code>ConfigRegistry.Player</code>	<code>PlayerBaseStatConfigSO</code>	Temel saldırı, HP, crit
<code>ConfigRegistry.Prestige</code>	<code>PrestigeConfigSO</code>	Min dalga, çarpan formülü
<code>ConfigRegistry.Schema</code>	<code>SchemaVersionSO</code>	Kayıt şema versiyonu
<code>ConfigRegistry.Upgrade</code>	<code>UpgradeNodeConfigSO[]</code>	Tüm yükseltme node'ları

Property	Tip	Ne için
ConfigRegistry.Run	RunConfigS0	Run parametreleri
ConfigRegistry.Realm	RealmIdentityConfigS0	Mevcut realm bilgisi
ConfigRegistry.RealmRegistry	RealmRegistryS0	Tüm realm'lerin listesi
ConfigRegistry.IsLoaded	bool	Config yüklenip yüklenmediği

Realm Değiştirme

```
await ConfigRegistry.BeginRealmSwapAsync(realmRegistry.GetPack("fire-realm"));  
// OnRealmSwapped tetiklenir – tüm sistemler cache'lerini yeniler
```

Önemli Kurallar

- OnConfigsLoaded tetiklenmeden önce config'lere erişmeyin — Editor build'de exception fırlatır.
- ConfigRegistry.IsLoaded ile yüklenip yüklenmediğini kontrol edebilirsiniz.
- Config değerlerini runtime'da değiştirmeyin; değişim için Realm Swap kullanın.

5. SaveService

Ne yapar: Oyun durumunu JSON olarak diske yazar/okur. Atomic yazma (yedek + rename), otomatik kayıt, şema versiyonlama ve migrasyon destekler.

Config: SchemaVersionSO

Create → Endless Engine → Schema Version

Alan	Açıklama	Başlangıç Değeri
CurrentSchemaVersion	Mevcut kayıt formatı numarası	1
MinimumCompatibleVersion	Uyumlu en düşük sürüm	1

Kullanım

```
// Yükle (Awake'te, async)
await _saveService.LoadAsync();

// Elle kaydet
await _saveService.SaveAsync();

// Otomatik kayıt (her X saniyede)
_saveService.StartAutoSave(intervalSeconds: 30);

// Olaylar
_saveService.OnSaveLoaded += (data, isNew) => { if (isNew) StartTutorial(); };
_saveService.OnSaveCompleted += (ok) => { if (!ok) ShowSaveError(); };
```

Kendi Sisteminizi Kayıt Etme

Her sistem `ISaveServiceProvider` arayüzünü uygular:

```
public class MySystem : MonoBehaviour, ISaveServiceProvider
{
    public int ProviderOrder => 50; // küçük = önce; Economy=10, Upgrade=20

    public void OnBeforeSave(SaveData data)
    {
        data.MySystemField = _internalValue;
    }

    public void OnAfterLoad(SaveData data)
    {
        _internalValue = data.MySystemField;
    }
}
```

Bootstrap'te kaydedin:

```
_saveService.RegisterServiceProvider(mySystem);
```

Kayıt Dosya Konumu

```
Windows : %USERPROFILE%\AppData\LocalLow\[Şirket]\[Oyun]\save_slot_0.json
Android : /data/data/[bundle_id]/files/save_slot_0.json
iOS : Application.persistentDataPath/save_slot_0.json
```

SaveProviderOrder Sabitler Listesi

Sistemlerin `OnBeforeSave` çağırılma sırası (küçük = önce):

Sabit	Değer	Yazan
Economy	10	CurrentResources
Currency	15	CurrencyBalances
UpgradeTree	20	UpgradeNodeStates
Ascension	25	AscensionCounts
Prestige	30	PrestigeCount
Inventory	35	InventoryItems
WaveAndCombat	40	WaveNumber
SkillTree	45	UnlockedSkillNodes
Generator	50	GeneratorStates
Building	55	PlacedBuildings
Click	60	ClickState
Pet	65	PetLevels
Zone	70	ZoneStates
UnlockLog	75	UnlockLogEntries
Milestone	80	CompletedMilestones
Statistics	85	StatisticsValues
Harvest	88	HarvestState
ClickLoop	89	ClickLoopState
Research	90	CompletedResearchNodes

Şema Migrasyonu

Yeni alan ekleyip eski kayıtlarla uyumluluğu korumak için:

```
Tools → Endless Engine → Schema Bump Utility → Bump Version
```

Bu araç `CurrentSchemaVersion`'ı artırır ve `Assets/Migrations/MigrationV{N}.cs` şablonu oluşturur.

6. EconomyService

Ne yapar: Oyunun ana para birimi (Altın) ledger'ı. Tüm kazanma/harcama buradan geçer. Hard cap uygular, çift harcamaya karşı korur.

Config: EconomyConfigSO

Create → Endless Engine → Economy Config

Alan	Açıklama	Örnek
ResourceHardCap	Maksimum bakiye	1e18
StartingGold	Yeni oyun başlangıç altını	0
NumberBackend	DoubleNumber veya BigDouble	DoubleNumber

Başlatma

```
_economyService.Initialize(
    upgradeTreeQuery: _upgradeTree, // IUpgradeTreeQuery
    saveNotifier: _saveService // ISaveNotifier
);
```

API

```
// Ekle
_economyService.AddResources(500.0);

// Çıkar (yetersizse sessizce yapar, log basar)
_economyService.DeductResources(100.0);

// Yükseltme satın al (maliyet UpgradeTree'den otomatik alınır)
_economyService.TryPurchase("dmg-boost-1");

// Bakiye
double balance = _economyService.CurrentResources;
long ballong = _economyService.CurrentResourcesLong;

// Static erişim (UI için kolaylık)
double b = EconomyService.CurrentResourcesStatic;
```

Olaylar

Olay	Parametre	Ne Zaman
OnResourcesChanged	(double current, double delta)	Her mutasyonda
OnUpgradePurchased	(string nodeId, double cost)	Başarılı alımda
OnPurchaseFailed	(string nodeId, double cost, double balance)	Yetersiz bakiyede

UI Bağlantısı

```
void Start()
{
    EconomyService.OnResourcesChanged += (current, _) =>
        goldText.text = FormatGold(current);
}

string FormatGold(double v)
{
    if (v >= 1e9) return $"{v/1e9:F1}B";
    if (v >= 1e6) return $"{v/1e6:F1}M";
    if (v >= 1e3) return $"{v/1e3:F1}K";
    return $"{v:F0}";
}
```

7. TickEngine

Ne yapar: Oyun saatidir. `TickIntervalSeconds` aralıklarla `OnTick` event'ini ateşler. Pasif gelir, araştırma sayacı, dalga sistemi gibi tüm zamanlı sistemler buraya abone olur.

Inspector Ayarları

Alan	Açıklama	Varsayılan
<code>TickIntervalSeconds</code>	Kaç saniyede bir tick	1.0
<code>TimeScale</code>	Hız çarpanı	1.0
<code>MaxTicksPerFrame</code>	Bir frame'de işlenecek maksimum birikmiş tick	5

API

```
// Tick'e abone ol
_tickEngine.OnTick += HandleTick;           // static Action<float>
// veya instance yöntemiyle:
_tickEngine.Subscribe(HandleTick);
_tickEngine.Unsubscribe(HandleTick);

void HandleTick(float deltaTime)
{
    // deltaTime = TickIntervalSeconds * TimeScale
    _myAccumulator += deltaTime;
}

// Durdur / devam ettir
_tickEngine.Pause();
_tickEngine.Resume();
bool paused = _tickEngine.IsPaused;

// Hız değiştir (ör. 2x hız boost satın alındı)
_tickEngine.TimeScale = 2.0f;

// Birikmiş süreyi sıfırla (prestige sonrası tick patlamasını önler)
_tickEngine.ResetAccumulator();

// Toplam etkin oyun süresi
float totalTime = _tickEngine.TotalEffectiveTime;
```

Testlerde Kullanım

```
// Manuel tick ateşle (EditMode testleri için)
TickEngine.FireTickForTesting(deltaTime: 1.0f);

// Abonelikleri temizle
TickEngine.ClearSubscribersForTesting();
```

Neden Her Şey TickEngine'e Bağlı?

`Update()` saniyede 60 kez çalışır. Pasif gelir gibi işlemlerin her frame hesaplanması gereksizdir. TickEngine bunları saniyede 1'e indirir; `MaxTicksPerFrame` oyuncu uzun süre kapattıktan sonra geri döndüğünde birikmiş tick'lerin bir frame'de patlamasını engeller.

BÖLÜM C

Gelir Sistemleri

8. GeneratorSystem

9. PassiveIncomeService

10. OfflineTimeCalculator

8. GeneratorSystem

Ne yapar: Oyuncunun satın alıp geliştirdiği pasif gelir kaynakları (fabrika, maden, çiftlik vb.). Her tick altın üretir.

Config: GeneratorConfigSO

Create → Endless Engine → Generator Config

Alan	Açıklama	Örnek
GeneratorId	Benzersiz ID	"coal-mine"
DisplayName	Gösterim adı	"Kömür Madeni"
BaseYieldPerSecond	Saniye başına temel kazanç	0.5
BaseCost	İlk satın alma fiyatı	10
CostScalingFactor	Her alımda maliyet çarpanı	1.15
MaxCount	Maksimum adet (0 = sınırsız)	0

Başlatma

```
_generatorSystem.Initialize(
    configs:      ConfigRegistry.Generators,
    economy:      _economyService,
    saveNotifier: _saveService
);
```

API

```
bool ok      = _generatorSystem.TryPurchase("coal-mine");
int  count   = _generatorSystem.GetCount("coal-mine");
double cost  = _generatorSystem.GetNextCost("coal-mine");
double totalYps = _generatorSystem.CalculateTotalYield();

GeneratorSystem.OnGeneratorPurchased += (id) => RefreshUI();
```

Toplu Satın Alma

```
// 10 adet satın al
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Ten);

// Maksimum satın al
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Max);

// Belirli adete kadar satın al
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Until, untilCount: 25);

if (result.Purchased > 0)
    Debug.Log($"{result.Purchased} adet alındı, {result.TotalCost:F0} altın harcadı");
```

Gelir Formülü

$$\text{gelir} = \text{BaseYieldPerSecond} \times \text{adet} \times \text{upgrade_çarpanı} \times \text{prestige_çarpanı}$$

`upgrade_çarpanı` UpgradeApplicationSystem tarafından `StatType.GeneratorSpeed` bazında hesaplanır.

9. PassiveIncomeService

Ne yapar: TickEngine'e abone olarak GeneratorSystem'in hesapladığı geliri her tick'te EconomyService'e ekler.

Başlatma

```
_passiveIncomeService.Initialize(  
    generators: _generatorSystem,  
    economy:    _economyService,  
    gameFlow:   _gameFlowStateMachine // null olabilir  
);
```

Her tick: `CalculateTotalYield() × dt → EconomyService.AddResources()` .

10. OfflineTimeCalculator

Ne yapar: Oyun kapalıyken geçen süreyi hesaplar ve karşılık gelen pasif geliri oyun açılınca `EconomyService` 'e ekler. Adı `OfflineTimeCalculator` 'dır — eski adı `OfflineProgressService` 'ti; o sınıf artık yoktur, kodda veya dokümanda kullanmayın.

Neden Initialize() Yok?

Eski versiyonda (v1.0.x) `Initialize(economy, generators, saveService)` çağırısı gerekiyordu. v1.1.0'da bu bağımlılıklar `SaveService.OnSaveLoaded` event'i üzerinden otomatik çözülür:

1. `OfflineTimeCalculator` `Awake()` 'te `SaveService.OnSaveLoaded` 'a abone olur.
2. `SaveService.LoadAsync()` tamamlanınca `OnSaveLoaded(saveData, isNewGame)` tetiklenir.
3. `OfflineTimeCalculator` bu event'i alır, `saveData.LastSaveTimestamp` ile şimdiki zamanı karşılaştırır.
4. Geçen süreyi `EconomyConfigSO.OfflineCapHours` ile sınırlar, `OfflineYieldMultiplier` ile çarpar.
5. Hesaplanan altını `EconomyService.AddResources()` ile ekler ve `OnOfflineGainCalculated` 'i ateşler.

Sonuç: Bootstrap'te `Initialize()` çağırması hem gereksizdir hem de derleme hatası verir — metod imzası yoktur.

EconomyConfigSO İlgili Alanlar

Alan	Açıklama	Varsayılan
<code>OfflineCapHours</code>	Maksimum offline kazanç süresi	8
<code>OfflineYieldMultiplier</code>	Offline gelir verimliliği (1.0 = tam, 0.5 = yarı)	0.5

Kurulum

```
// Bootstrap'te yapmanız gereken SADECE budur:
// Hierarchy → Services → OfflineTimeCalculator (MonoBehaviour olarak ekleyin)
//
// YANLIŞ (derleme hatası):
// _offlineCalc.Initialize(_economyService, _generatorSystem, _saveService); // ← YOK
//
// DOĞRU:
// Hiçbir şey çağırmayın. SaveService.OnSaveLoaded → OfflineTimeCalculator zinciri
// Awake() içinde otomatik kurulur.
```

Dikkat: `OfflineTimeCalculator` sahnede bulunmalı ve `SaveService` ile aynı sahnede olmalıdır. Farklı sahnelerdeyse `Awake()` sırası garanti edilmez; bu durumda `[DefaultExecutionOrder]` kullanın.

Hesaplama Formülü

```
offlineSüresi = min(şimdi - lastSaveTimestamp, OfflineCapHours × 3600)
offlineKazanç = GeneratorSystem.CalculateTotalYield()
                × offlineSüresi
                × OfflineYieldMultiplier
```

`isNewGame == true` ise hesaplama atlanır — yeni oyunda geçmiş kayıt yoktur.

UI'da Gösterme

```
// Herhangi bir MonoBehaviour'da abone ol
OfflineTimeCalculator.OnOfflineGainCalculated += (gold, seconds) =>
{
    if (gold > 0)
        offlinePopup.Show(
            $"{FormatGold(gold)} kazandınız",
            $"{FormatTime(seconds)} çevrimdışıydınız"
        );
};
```

Testlerde Kullanım

```
// Sahte SaveData ile hesabı manuel tetikle (EditMode testi)
var fakeSave = new SaveData();
fakeSave.LastSaveTimestamp = DateTime.UtcNow.AddHours(-3); // 3 saat önce kaydedilmiş
_offlineCalc.InvokeForTesting(fakeSave, isNewGame: false);

// Single-fire guard'ı sıfırla (birden fazla test senaryosu için)
_offlineCalc.ResetForTesting();
```

BÖLÜM D

Aktif Oynanış Sistemleri

11. ClickYieldService

12. ClickLoopService

13. HarvestLoopService

11. ClickYieldService

Ne yapar: Oyuncunun ekrana her tıkladığında aldığı anlık altın miktarını hesaplar. Basit clicker oyunları için kullanılır.

Not: Sahne içinde ClickLoopService da varsa ikisini aynı anda **kullanmayın** — Bootstrap otomatik uyarı verir ve ClickYieldService'i devre dışı bırakır. Birini seçin.

Başlatma

```
_clickYieldService.Initialize(  
    config:          myClickSourceConfig,    // ClickSourceConfigS0  
    economy:         _economyService,  
    passiveYieldGetter: null                // YieldRateClickFraction için opsiyonel  
);  
_clickYieldService.SetInputProvider(_inputProvider);
```

API

```
// Tıklama gelirini işle (buton OnClick'e bağlayın)  
_clickYieldService.ProcessClick();  
  
// Tıklama başına kazanç (UI için)  
double perClick = _clickYieldService.GetClickYield();
```

12. ClickLoopService

Ne yapar: Dünyada yerleştirilen `ClickTarget` nesnelere tıklanarak hasar verilir; hedef yok edilince altın ödülü verilir ve belirli süre sonra yeniden doğar. Combo çarpanı, kritik vuruş ve oto-tıklama içerir.

Sistem Bileşenleri

Dosya	Görev
<code>ClickLoopConfigSO</code>	Combo/crit/oto-tıklama parametreleri
<code>ClickTargetConfigSO</code>	Tek bir hedef türünün HP/yield/respawn değerleri
<code>ClickTarget</code>	Sahnedeki tıklanabilir nesne (MonoBehaviour)
<code>ClickTargetRegistry</code>	Sahnedeki tüm aktif hedeflerin listesi
<code>ClickLoopService</code>	Ana servis; tıklamaları algılar, hasar uygular, gelir verir
<code>ClickLoopOfflineCalculator</code>	Çevrimdışı süre için oto-tıklama geliri hesabı
<code>ClickLoopHUDController</code>	Combo çubuğu, crit flash, HP barı

Config: ClickLoopConfigSO

Create → Endless Engine → Click Loop → Click Loop Config

Alan	Açıklama	Örnek
<code>ComboDecayDelay</code>	Sonraki tıklamaya kadar bekleme süresi (saniye)	1.5
<code>ComboDecayRate</code>	Saniyede azalan combo puanı	8
<code>MaxComboMultiplier</code>	Maksimum combo çarpanı	8
<code>ComboPointsPerStep</code>	Çarpan başına gereken combo puanı	5
<code>BaseCritChance</code>	Temel kritik şans (0-1)	0.05
<code>BaseCritMultiplier</code>	Kritik hasar çarpanı	3
<code>BaseAutoClickRate</code>	Saniyede otomatik tıklama sayısı	0
<code>OfflineCapHours</code>	Maks. offline hesap süresi	8
<code>OfflineEfficiency</code>	Offline gelir verimliliği	0.25

Config: ClickTargetConfigSO

Create → Endless Engine → Click Loop → Click Target Config

Alan	Açıklama	Örnek
<code>TargetId</code>	Benzersiz ID	"coin-bag"
<code>MaxHP</code>	Hedefin maksimum canı	10

Alan	Açıklama	Örnek
DamagePerClick	Tıklama başına temel hasar	1
BaseYield	Yok edilince verilen temel altın	50
AwardYieldPerClick	Her tıklamada kısmi altın ver	false
RespawnSeconds	Yeniden doğma süresi	5
ComboContribution	Bu tıklamanın combo'ya katkısı	1
DestructionVFXPrefab	Yok edilince oynatılacak efekt	(opsiyonel)

ClickTarget Sahneye Ekleme

1. Yeni bir GameObject oluşturun.
2. ClickTarget component'ini ekleyin (BoxCollider2D otomatik eklenir — [RequireComponent] garantisi).
3. Inspector'da _config alanına ClickTargetConfigSO asset'ini atayın.
4. GameObject'e uygun bir Layer atayın (ör. ClickableTargets).

```
Hierarchy'de:
CoinBag
├─ Sprite (görsel çocuk)
├─ BoxCollider2D (otomatik)
└─ ClickTarget (siz eklersiniz)
```

Toplu Yerleştirme: ClickTargetSpawner

Birden fazla hedefi otomatik oluşturmak için:

```
// ClickTargetSpawner component'i
[SerializeField] private SpawnPoint[] _spawnPoints;

// SpawnPoint yapısı
[Serializable]
public struct SpawnPoint
{
    public ClickTargetConfigSO Config;
    public Vector3 WorldPosition;
}
```

SpawnPoints listesini Inspector'da doldurun; Start() 'ta otomatik spawn eder.

Başlatma

```
_clickLoopService.Initialize(
    config: _clickLoopConfig,
    economy: _economyService,
    input: _inputProvider,
    targetLayer: LayerMask.GetMask("ClickableTargets"),
    statistics: _statisticsService, // opsiyonel
    vfx: _vfxController // opsiyonel
);
```

Olaylar

```
// Hedefe tıklandı
_clickLoopService.OnTargetClicked += (target, damage, gold, wasCrit) =>
{
    if (wasCrit) PlayCritEffect();
    ShowGoldPop(gold, target.WorldPosition);
};

// Hedef yok edildi
_clickLoopService.OnTargetDestroyed += (target) =>
{
    PlayDestroyEffect(target.WorldPosition);
};

// Combo değişti
_clickLoopService.OnComboChanged += (multiplier) =>
{
    comboText.text = $"x{multiplier:F1}";
};

// Kritik vuruş
_clickLoopService.OnCrit += (critMult) =>
{
    StartCritFlash();
};
```

HUD Bağlantısı

```
_clickLoopHUD.Initialize(_clickLoopService, _clickLoopConfig);
```

ClickLoopHUDController Inspector alanları:

Alan	Görev
_comboBar	Combo doluluk çubuğu (Slider/Image)
_comboText	"x2.5" gibi metin
_critFlash	Kritik vuruşta yanıp sönen Image
_targetHPBar	Aktif hedefin HP barı
_yieldPopText	Kazanılan altın pop-up metni

Offline Hesaplama

ClickLoopOfflineCalculator sahnede kaç tür hedef varsa onların ClickTargetConfigS0 array'ini alır; istatistik servisi değil.

```
// Bootstrap'te:
_clickLoopOfflineCalc.Initialize(
    config: _clickLoopConfig,
    economy: _economyService,
    targetConfigs: _clickTargetConfigs // ClickTargetConfigS0[] - sahnedeki hedef türleri
);
_saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;

// OnDestroy'da:
_saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
```

Event:

```
ClickLoopOfflineCalculator.OnOfflineClickGainCalculated += (gold, seconds) =>
    offlinePopup.Show($"Oto-tıklama: {FormatGold(gold)} {FormatTime(seconds)}");
```

Upgrade Bağlantıları (StatType Enum)

UpgradeNodeConfigSO'da AffectedStat olarak şu değerleri kullanabilirsiniz:

StatType	Etkisi
ClickDamage	Tıklama başına hasar
ClickTargetMaxHP	Hedef maksimum canı
ClickTargetRespawnRate	Yeniden doğma hızı
ClickYieldMultiplier	Altın kazanç çarpanı
ClickComboMultiplier	Maksimum combo çarpanı
ClickComboDecayRate	Combo azalma hızı
ClickCritChance	Kritik şans artışı
ClickCritMultiplier	Kritik hasar çarpanı
ClickAutoRate	Otomatik tıklama hızı

Kayıt / Yükleme

```
// Bootstrap'te:
_saveService.RegisterServiceProvider(_clickLoopService);
// ClickLoopService otomatik olarak OnBeforeSave/OnAfterLoad yönetir.
// Her ClickTarget'ın respawn durumu ve timer'ı SaveData.ClickLoopState içinde saklanır.
```

13. HarvestLoopService

Ne yapar: Oyuncunun cursoru/parmağı dünya nesnelerinin (HarvestNode) üzerinde tutularak devamlı hasar verilir. Cursoru bir alanda gezdirmeye, tüm temas eden node'ları eşzamanlı hasar verme, combo ve çevrimdışı ilerleme içerir.

Sistem Bileşenleri

Dosya	Görev
HarvestAreaConfigSO	Cursor yarıçapı, tick hızı, combo parametreleri
HarvestNodeConfigSO	Tek bir node türünün HP/yield/respawn değerleri
HarvestNode	Sahnedeki hasat edilebilir nesne (MonoBehaviour)
HarvestNodeRegistry	Sahnedeki tüm aktif node'ların listesi
HarvestCursor	Cursor konumunu takip eden ve alan tespitini yapan servis
HarvestLoopService	Ana servis; tick'te hasar uygular, gelir verir
HarvestOfflineCalculator	Çevrimdışı süre için hasat geliri hesabı
HarvestHUDController	Combo çubuğu, yield pop, cursor radius göstergesi

Config: HarvestAreaConfigSO

Create → Endless Engine → Harvest → Harvest Area Config

Alan	Açıklama	Örnek
BaseRadius	Cursor'ın başlangıç etki yarıçapı (dünya birimi)	1.5
BaseTickInterval	Kaç saniyede bir hasar tick'i	0.25
ComboDecayDelay	Sonraki hit'e kadar bekleme süresi	1.5
ComboDecayRate	Saniyede azalan combo puanı	8
MaxComboMultiplier	Maksimum combo çarpanı	8
ComboPointsPerStep	Çarpan başına gereken puan	5
OfflineCapHours	Maks. offline hesap süresi	8
OfflineEfficiency	Offline gelir verimliliği	0.3

Config: HarvestNodeConfigSO

Create → Endless Engine → Harvest → Harvest Node Config

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"gold-ore"
MaxHP	Node'un maksimum canı	20

Alan	Açıklama	Örnek
DamagePerTick	Tick başına temel hasar	2
BaseYield	Tüketilince verilen temel altın	100
YieldPerTickRatio	Her tick'te BaseYield'in kaçta biri verilir	0.05
RespawnSeconds	Yeniden doğma süresi	10
ComboContribution	Her hit'in combo'ya katkısı	1
DestructionVFXPrefab	Tüketilince oynatılacak efekt	(opsiyonel)

HarvestNode Sahneye Ekleme

1. Yeni bir GameObject oluşturun.
2. `HarvestNode` component'ini ekleyin (`Collider2D` otomatik eklenir).
3. Inspector'da `_config` alanına `HarvestNodeConfigSO` asset'ini atayın.
4. Node'u uygun bir Layer'a atayın (ör. `HarvestNodes`).

```
Hierarchy'de:
GoldOre
├─ Sprite (görsel çocuk)
├─ CircleCollider2D (otomatik)
└─ HarvestNode (siz eklersiniz)
```

HarvestCursor Kurulumu

1. Sahne'de bir GameObject oluşturun.
2. `HarvestCursor` component'ini ekleyin.
3. Inspector'da `_config` (`HarvestAreaConfigSO`) ve `_harvestLayer` (`LayerMask`) atayın.
4. Bootstrap'te InputProvider'ı enjekte edin:

```
_harvestCursor.Inject(_inputProvider);
```

`HarvestCursor` her frame cursor dünya konumunu alır ve `Physics2D.OverlapCircleNonAlloc` ile temas eden node'ları tespit eder.

Başlatma

```
_harvestLoopService.Initialize(
    cursor: _harvestCursor,
    config: _harvestAreaConfig,
    economy: _economyService,
    statistics: _statisticsService, // opsiyonel
    vfx: _vfxController // opsiyonel
);
```

Olaylar

```
// Node'a hasar verildi
_harvestLoopService.OnNodeDamaged += (node, damage, gold) =>
{
    ShowGoldPop(gold, node.WorldPosition);
};

// Node tükendi
_harvestLoopService.OnNodeDepleted += (node) =>
{
    PlayDepletionEffect(node.WorldPosition);
};

// Gelir ödendi (tick bazında toplam)
_harvestLoopService.OnYieldAwarded += (totalGold) =>
{
    // Tick başına toplam kazanç
};

// Combo değişti
_harvestLoopService.OnComboChanged += (multiplier) =>
{
    comboBar.fillAmount = (multiplier - 1f) / (maxCombo - 1f);
};
```

HUD Bağlantısı

```
_harvestHUD.Initialize(_harvestLoopService, _harvestCursor, _harvestAreaConfig);
```

HarvestHUDController Inspector alanları:

Alan	Görev
_comboBar	Combo doluluk çubuğu
_comboText	"x2.5" metni
_yieldPopText	Kazanılan altın pop-up
_cursorRadiusIndicator	Cursor'ın etki alanını gösteren daire UI
_pxPerWorldUnit	UI→Dünya birimi dönüşüm katsayısı

Offline Hesaplama

HarvestOfflineCalculator sahnedeki node türlerinin HarvestNodeConfigS0 array'ini alır; istatistik servisi değil.

```
// Bootstrap'te:
_harvestOfflineCalc.Initialize(
    config: _harvestAreaConfig,
    economy: _economyService,
    nodeConfigs: _harvestNodeConfigs // HarvestNodeConfigS0[] - sahnedeki node türleri
);
_saveService.OnSaveLoaded += _harvestOfflineCalc.HandleSaveLoaded;

// OnDestroy'da:
_saveService.OnSaveLoaded -= _harvestOfflineCalc.HandleSaveLoaded;
```

Event:

```
HarvestOfflineCalculator.OnOfflineHarvestCalculated += (gold, seconds) =>
    offlinePopup.Show($"Hasat: {FormatGold(gold)} ({FormatTime(seconds)})");
```

Upgrade Bağlantıları (StatType Enum)

StatType	Etkisi
HarvestRadius	Cursor etki yarıçapı
HarvestTickRate	Tick hızı
HarvestYieldMultiplier	Altın kazanç çarpanı
HarvestNodeMaxHP	Node maksimum canı
HarvestNodeRespawnRate	Node yeniden doğma hızı
HarvestComboMultiplier	Maksimum combo çarpanı
HarvestComboDecayRate	Combo azalma hızı
HarvestMultiNodeBonus	Aynı anda birden fazla node vurmanın bonusu

Kayıt / Yükleme

```
// Bootstrap'te:
_saveService.RegisterServiceProvider(_harvestLoopService);
// Her HarvestNode'un respawn durumu SaveData.HarvestState içinde saklanır.
```

ClickLoop ile HarvestLoop Karşılaştırması

Özellik	ClickLoop	HarvestLoop
Tetikleyici	Tıklama/dokunma	Cursor üzerinde tutma
Hedef	Tek nesne	Alan içindeki tüm nesneler
Hasar zamanlaması	Anlık	Tick bazlı
Zorluk	Aktif tıklama gerekir	Sürükleme/tutma
İdeal tür	Mobil tap, masaüstü click	Masaüstü mouse, tablet drag

BÖLÜM E

İlerleme Sistemleri

14. UpgradeTreeService

15. UpgradeApplicationSystem

16. SkillTreeService

17. ResearchService

18. PrestigeStateManager

19. AscensionStateManager

14. UpgradeTreeService

Ne yapar: DAG (yönlü asiklik çizge) tabanlı yükseltme ağacı. Önkoşullu düğümler, rank sistemi, ağırlıklı kart çekimi.

Config: UpgradeNodeConfigSO

Create → Endless Engine → Upgrade Node Config

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"dmg-1"
DisplayName	Görüntülenecek ad	"Hasar I"
BaseCost	Temel altın maliyeti	100
CostScalingFactor	Her rank'ta maliyet çarpanı	1.5
MaxRank	Maksimum seviye (0 = sınırsız)	5
AffectedStat	Etkilediği stat (StatType enum)	Damage
EffectPerRank	Rank başına etki	0.1
EffectType	FlatBonus veya PercentBonus	PercentBonus
PrerequisiteNodeIDs	Önce açılması gereken düğümler	["dmg-basic"]
PrestigeGateRequirement	Erişim için gereken prestige sayısı	0
SelectionWeight	Kart çekimindeki görünme olasılığı	10

Tüm StatType Değerleri

Bu enum değerleri UpgradeNodeConfigSO.AffectedStat alanında kullanılır:

Savaş: Damage · AttackInterval · AttackRange · CritChance · CritMultiplier · AreaDamage

Sağlık: MaxHP · MoveSpeed · DamageReduction · HPRegen

Ekonomi: GoldDropMultiplier · GoldPickupRange · BonusRunReward · ComboMultiplier

Üretim: IdleYieldRate · GeneratorSpeed · OfflineYieldRate · ActiveRunPassiveBonus

Prestige: PrestigeMultiplier · StartingGoldBonus · RunDurationBonus · DoubleGeneratorChance

Hasat Loopuna Özel (8 adet): HarvestRadius · HarvestTickRate · HarvestYieldMultiplier · HarvestNodeMaxHP · HarvestNodeRespawnRate · HarvestComboMultiplier · HarvestComboDecayRate · HarvestMultiNodeBonus

Click Loopuna Özel (9 adet): ClickDamage · ClickTargetMaxHP · ClickTargetRespawnRate · ClickYieldMultiplier · ClickComboMultiplier · ClickComboDecayRate · ClickCritChance · ClickCritMultiplier · ClickAutoRate

Başlatma

UpgradeTreeService'in Initialize() metodu yoktur. Bunun yerine HandleConfigsLoaded() çağrılır:

```
// ConfigRegistry yüklendikten SONRA çağırın
_upgradeTree.HandleConfigsLoaded(); // ConfigRegistry.Upgrades'i okur
_saveService.RegisterStateProvider(_upgradeTree);
```

Veya Addressables akışında:

```
ConfigRegistry.OnConfigsLoaded += () => _upgradeTree.HandleConfigsLoaded();
```

API

```
// Rank sorgula
int rank = _upgradeTree.GetNode("dmg-1").CurrentRank;

// Satın alınabilir mi
bool canBuy = _upgradeTree.IsNodeAvailable("dmg-1");

// Maliyet
long cost = _upgradeTree.GetNodeCost("dmg-1");

// Satın al (EconomyService.TryPurchase içinden otomatik çağrılır)
_economyService.TryPurchase("dmg-1");

// Kart çekimi (dalga sonu ekran için)
var cards = _upgradeTree.GetAvailableNodes(); // tüm açık düğümler
```

Görsel Düzenleme

Tools → Endless Engine → Upgrade Tree Editor

15. UpgradeApplicationSystem

Ne yapar: Tüm stat hesaplarının merkezi. Run-scope (geçici) ve kalıcı efektleri katmanlayarak her `StatType` için efektif değer hesaplar ve cache'ler. Herhangi bir sisteme bağımlı olmaksızın static olarak çalışır.

Önemli: Bu bir `static class` 'tır — instance oluşturulmaz, `new` veya `MonoBehaviour` gerekmez, sahnede `GameObject` gerekmez. Sadece namespace'i ekleyin ve doğrudan sınıf adıyla kullanın.

Stat Formülü

```
efektif = (base + Σ AdditiveFlat efektler)
          × (1 + Σ AdditivePercent efektler)
          × prestige_permanent_multiplier
          × ascension_cascade_multiplier
```

Örnek: Damage stat'ı, base=10, 2 tane %20 PercentBonus upgrade, prestige 1.5×

```
efektif = 10 × (1 + 0.20 + 0.20) × 1.5 = 10 × 1.40 × 1.5 = 21.0
```

Cache ve Dirty Flag Sistemi

`GetEffectiveStat()` her çağrıda yeniden hesaplamaz — cache kullanır:

- `ApplyUpgradeEffect()` çağrıldığında ilgili stat "dirty" işaretlenir.
- Bir sonraki `GetEffectiveStat()` çağrısında dirty stat yeniden hesaplanır ve cache güncellenir.
- Tekrar çağrıda (cache-hit) sıfır allocation, sıfır hesaplama yükü.

Bu yüzden `Update()` içinde her frame `GetEffectiveStat()` çağrısı güvenlidir.

Run-Scope vs Kalıcı Efektler

Tür	Ne Zaman Silinir	Nasıl Eklenir
Run-scope	<code>ClearRunEffects()</code> ile (prestige/run sonu)	<code>ApplyUpgradeEffect(..., isPermanent: false)</code>
Kalıcı	Asla — sadece <code>SetPermanentMultiplier</code> günceller	<code>ApplyUpgradeEffect(..., isPermanent: true)</code>

`UpgradeTreeService` bir node satın alındığında `isPermanent: false` efekt uygular — run bitince `ClearRunEffects()` bunları temizler. Prestige kalıcı çarpanı `SetPermanentMultiplier` ile ayrıca set edilir.

API

```
// Namespace: EndlessEngine.Core
using EndlessEngine.Core;

// Efektif değeri oku (cache-hit = sıfır allocation)
float damage = UpgradeApplicationSystem.GetEffectiveStat(StatType.Damage);
float radius = UpgradeApplicationSystem.GetEffectiveStat(StatType.HarvestRadius);
float crit = UpgradeApplicationSystem.GetEffectiveStat(StatType.ClickCritChance);
float autoRate = UpgradeApplicationSystem.GetEffectiveStat(StatType.ClickAutoRate);

// Stat değişince tetiklenen event (UI preview için)
UpgradeApplicationSystem.OnEffectiveStatChanged += (stat, value) =>
{
    if (stat == StatType.Damage)
        damagePreviewText.text = $"Hasar: {value:F1}";
};

// Efekt uygula (genellikle UpgradeTreeService otomatik çağırır – elle çağırmak nadiren gerekir)
UpgradeApplicationSystem.ApplyUpgradeEffect(
    stat: StatType.Damage,
    magnitude: 0.2f,
    effectType: EffectType.AdditivePercent,
    isPermanent: false // run-scope
);

// Run/Prestige sonrası geçici efektleri temizle (kalıcılar korunur)
UpgradeApplicationSystem.ClearRunEffects();
// → Prestige akışında OnPrestigeStarted event'ine abone olun:
PrestigeStateManager.OnPrestigeStarted += () =>
    UpgradeApplicationSystem.ClearRunEffects();

// Prestige kalıcı çarpanını güncelle (PrestigeStateManager bunu otomatik çağırır)
UpgradeApplicationSystem.SetPermanentMultiplier(1.5f);

// Satın almadan önce etkiyi simüle et (upgrade preview UI için)

float current = UpgradeApplicationSystem.GetEffectiveStat(StatType.Damage);
float simulated = UpgradeApplicationSystem.SimulateEffect("dmg-1", additionalRanks: 1);
previewText.text = $"{{current:F1}} → {{simulated:F1}}";
```

Hangi Sistemler Bunu Kullanır?

Tüm stat-bağımlı sistemler doğrudan bu API üzerinden değer okur — kendi içlerinde stat hesaplamazlar:

Sistem	Hangi Stat'ları Okur
AutoBattleController	Damage, AttackInterval, CritChance, CritMultiplier
ClickLoopService	ClickDamage, ClickCritChance, ClickAutoRate, ClickYieldMultiplier
HarvestLoopService	HarvestRadius, HarvestTickRate, HarvestYieldMultiplier
PassiveIncomeService	GeneratorSpeed, IdleYieldRate
OfflineTimeCalculator	OfflineYieldRate
BaseStatUpgradeProvider	Damage, AttackInterval, CritChance, CritMultiplier, MoveSpeed

Testlerde Sıfırlama

```
[TearDown]
public void TearDown()
{
    UpgradeApplicationSystem.ResetForTesting(); // efektler, cache, dirty flag, multiplier
}
```

16. SkillTreeService

Ne yapar: Prestige ile kazanılan beceri puanları karşılığı açılan kalıcı beceri ağacı. UpgradeTree'nin aksine Skill Tree prestige'den sonra **sıfırlanmaz** — kazanımlar kalıcıdır. Her node belirli bir StatType 'ı etkiler; GetAllActiveEffects() çıktısı UpgradeApplicationSystem 'a beslenir.

v1.0.x'ten Fark: Başlatma

Eski versiyonda config ConfigRegistry.SkillTreeConfigs 'ten alınıyordu:

```
// ESKİ (v1.0.x):
_skillTree.Initialize(configs: ConfigRegistry.SkillTreeConfigs, startingPoints: 0);
```

v1.1.0'da config'ler doğrudan array olarak geçirilir — Registry üzerinden değil. Bu kasıtlı bir değişiklik: skill tree config'leri realm değişikliğinde değişmez, oyuncuya özgü kalır, bu yüzden ConfigRegistry'de tutulmaz.

Config: SkillTreeConfigSO ve SkillNodeConfigSO

```
Create → Endless Engine → Skill Tree Config (ağaç kapsayıcı)
Create → Endless Engine → Skill Node Config (her beceri düğümü)
```

SkillTreeConfigSO , SkillNodeConfigSO referanslarını listeler. Her node:

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"skill-crit-1"
DisplayName	Gösterim adı	"Kritik Uzmanı I"
SkillPointCost	Açmak için gereken puan	1
AffectedStat	Etkilenen stat (StatType)	CritChance
EffectPerRank	Her ranktaki etki	0.05
IsRefundable	Geri alınabilir mi?	true
PrerequisiteNodeIDs	Önce açılması gereken node'lar	["skill-basic"]

Başlatma

```
// Config'ler doğrudan array olarak geçirilir — ConfigRegistry üzerinden değil
_skillTree.Initialize(
    trees: new SkillTreeConfigSO[] { _combatSkillTree, _economySkillTree },
    startingPoints: 0 // sıfırdan başla; prestige ödülüyle AddPoints() çağrılır
);
_saveService.RegisterStateProvider(_skillTree);
// SaveProviderOrder.SkillTree = 45
```

API

```
// Mevcut puan
int points = _skillTree.SkillPoints;

// Puan ekle (prestige ödülü olarak)
_skillTree.AddPoints(amount: 1);

// Beceri aç
bool ok = _skillTree.TryUnlock("combat-tree", "skill-crit-1");

// Beceri geri al (puan iadesi)
bool refunded = _skillTree.TryRefund("combat-tree", "skill-crit-1");

// Açık mı kontrol et
bool isOpen = _skillTree.IsUnlocked("combat-tree", "skill-crit-1");

// Tüm aktif efektleri al – UpgradeApplicationSystem'a beslemek için
IReadOnlyList<SkillEffect> effects = _skillTree.GetAllActiveEffects();

// Olaylar
SkillTreeService.OnNodeUnlocked += (treeId, nodeId) => RefreshSkillUI();
SkillTreeService.OnNodeRefunded += (treeId, nodeId) => RefreshSkillUI();
SkillTreeService.OnSkillPointsChanged += (points) => UpdatePointsText(points);
SkillTreeService.OnUnlockFailed += (treeId, nodeId, reason) =>
    ShowError($"Açılamadı: {reason}");
// reason: AlreadyUnlocked | InsufficientPoints | PrerequisiteNotMet | NodeNotFound
```

GetAllActiveEffects() → UpgradeApplicationSystem Entegrasyonu

Skill tree node'larının efektleri otomatik olarak stat sistemine uygulanmaz — siz uygularsınız. Tipik kalıp: bir node açıldığında veya yükleme sonrasında tüm aktif efektleri sisteme besleyin:

```
void ApplyAllSkillEffects()
{
    // Önce eski skill efektlerini temizle (isPermanent: true olanlar hariç)
    // Not: ClearRunEffects kalıcı efektleri silmez – sadece run-scope efektleri siler
    // Skill efektleri kalıcı olduğu için ayrı bir iz tutmanız gerekebilir.
    // En güvenli yaklaşım: tüm kalıcı efektleri prestige sonrası yeniden uygulamak:

    foreach (var effect in _skillTree.GetAllActiveEffects())
    {
        UpgradeApplicationSystem.ApplyUpgradeEffect(
            stat: effect.AffectedStat,
            magnitude: effect.Magnitude,
            effectType: effect.EffectType,
            isPermanent: true // skill efektleri prestige'den sonra da kalır
        );
    }
}

// Çağırılacak yerler:
// 1. Yükleme sonrası: SaveService.OnSaveLoaded += (_, __) => ApplyAllSkillEffects();
// 2. Node açıldıktan sonra: SkillTreeService.OnNodeUnlocked += (_, __) => ApplyAllSkillEffects();
// 3. Node iade sonrası: SkillTreeService.OnNodeRefunded += (_, __) => ApplyAllSkillEffects();
```

Hata Kodları

```
// TryUnlock başarısız olursa OnUnlockFailed tetiklenir:  
// SkillUnlockFailReason.AlreadyUnlocked      - zaten açık  
// SkillUnlockFailReason.InsufficientPoints    - yeterli puan yok  
// SkillUnlockFailReason.PrerequisiteNotMet    - önkoşul node henüz açılmamış  
// SkillUnlockFailReason.NodeNotFound         - treeId/nodeId yanlış  
  
// TryRefund başarısız olursa (event yok, sadece false döner):  
// SkillRefundFailReason.NotUnlocked          - zaten kapalı  
// SkillRefundFailReason.NotRefundable        - IsRefundable = false  
// SkillRefundFailReason.NodeNotFound         - yanlış ID  
// SkillRefundFailReason.HasDependents        - bu node'a bağımlı başka açık node var
```

Görsel Düzenleme

Tools → Endless Engine → Skill Tree Editor

17. ResearchService

Ne yapar: Zaman bazlı araştırma kuyruğu. Araştırma başlatılır, X saniye/tick sonra tamamlanır ve kalıcı bonus aktive edilir.

Config: ResearchTreeConfigSO ve ResearchNodeConfigSO

Create → Endless Engine → Research → Research Tree Config
Create → Endless Engine → Research → Research Node Config

Alan	Açıklama
ResearchId	Benzersiz ID
DurationTicks	Kaç tick sürer
Cost	Altın maliyeti
AffectedStat	Tamamlanınca etkilenen stat
EffectValue	Etki miktarı

Başlatma

```
_researchService.Initialize(
    trees:          ConfigRegistry.ResearchTrees,
    economyService: _economyService,
    currencyService: null // ikincil para birimi için opsiyonel
);
_tickEngine.OnTick += _researchService.OnTick;
_saveService.RegisterStateProvider(_researchService);
```

API

```
// Kuyruğa ekle
bool ok = _researchService.EnqueueNode("tech-tree", "fire-upgrade");

// Tamamlandı mı
bool done = _researchService.IsCompleted("tech-tree", "fire-upgrade");

// Kuyrukta mı
bool queued = _researchService.IsQueued("tech-tree", "fire-upgrade");

// Olaylar
ResearchService.OnNodeCompleted += (treeId, nodeId) =>
    ShowToast($"Araştırma tamamlandı: {nodeId}");

ResearchService.OnResearchProgress += (treeId, nodeId, done, total) =>
    progressBar.fillAmount = (float)done / total;
```

18. PrestigeStateManager

Ne yapar: "Yumuşak sıfırlama" sistemi. Altın, dalga ilerlemesi ve run-scope yükseltme efektlerini sıfırlar; karşılığında kalıcı gelir çarpanı (`UpgradeApplicationSystem.SetPermanentMultiplier`) kazanılır. `IPrestigeQuery` arayüzünü uygular — diğer sistemler prestige sayısını bu arayüzden okur.

Neden Initialize() Yok? (v1.0.x'ten Fark)

Eski versiyonda çağrı şuydu:

```
// ESKİ (v1.0.x) – ARTIK GEÇERSİZ, derlenmez:
_prestigeManager.Initialize(
    economyService: _economyService,
    waveManager: _waveManager,
    upgradeTree: _upgradeTree,
    saveService: _saveService
);
```

v1.1.0'da bu bağımlılıkların tamamı kaldırıldı çünkü:

- `EconomyService` prestige akışında `OnPrestigeStarted` event'ini dinleyerek kendi altınını sıfırlar — doğrudan referans gerekmez.
- `WaveSpawnManager` prestige sonrası `ResetForNewRun()` 'ı aynı event'le çalıştırır.
- `UpgradeTreeService` prestige event'ine abone olup run-scope efektleri temizler.
- `PrestigeStateManager` config'i `ConfigRegistry.Prestige` 'den `OnEnable()` 'da okur; ayrıca enjeksiyona gerek kalmaz.

Sonuç: Bootstrap'te tek yapmanız gereken `ConfigRegistry` 'ye `PrestigeConfigSO` eklemek ve `RegisterstateProvider` çağırmasıdır.

Config: PrestigeConfigSO

Create → Endless Engine → Prestige Config

Alan	Açıklama	Örnek
<code>MinWaveToPrestige</code>	Prestige için gereken minimum dalga numarası	10
<code>BaseMultiplierPerPrestige</code>	Her prestige'de kazanılan kalıcı çarpan artışı	0.1
<code>MultiplierFormula</code>	Artış formülü: <code>Linear</code> / <code>Exponential</code> / <code>Logarithmic</code>	Linear
<code>MaxPermanentMultiplier</code>	Maksimum kalıcı çarpan üst sınırı (0 = sınırsız)	0

Formül örnekleri (`BaseMultiplierPerPrestige` = 0.1 ile 5. prestige'de):

- **Linear:** $1.0 + 5 \times 0.1 = 1.5\times$
- **Exponential:** $1.0 \times (1 + 0.1)^5 \approx 1.61\times$
- **Logarithmic:** $1.0 + 0.1 \times \ln(5+1) \approx 1.18\times$

Kurulum (Initialize()) ÇAĞRILMAZ

```
// ADIM 1: ConfigRegistry'ye prestige config ekleyin.
// Addressables akışında bu otomatiktir (ConfigLoadingService yapar).
// Test veya doğrudan sahne bootstrap'inde:
ConfigRegistry.InjectForTesting(
    economy: _economyConfig,
    wave: _waveConfig,
    prestige: _prestigeConfig, // ← PrestigeStateManager OnEnable'da bunu okur
    schema: _schemaVersion
);

// ADIM 2: SaveService'e kaydedin.
_saveService.RegisterStateProvider(_prestigeManager);

// ADIM 3: WaveSpawnManager → PrestigeStateManager köprüsü.
// PrestigeStateManager CanPrestige kontrolü için mevcut dalga numarasına ihtiyaç duyar.
// WaveSpawnManager her yeni dalgada OnWaveStarted event'i ateşler;
// siz bu event'te prestige manager'a dalga numarasını bildirmelisiniz:
WaveSpawnManager.OnWaveStarted += (wave) => _prestigeManager.SetCurrentWave(wave);

// YANLIŞ (DERLEME HATASI):
// _prestigeManager.Initialize(...); // ← bu metod yoktur
```

Prestige Akışı (Adım Adım)

1. UI'dan TryPrestige() çağrılır
2. CanPrestige kontrolü:
 - SetCurrentWave() ile bildirilen dalga >= MinWaveToPrestige mi?
 - Devam eden bir prestige animasyonu var mı? (re-entrant guard)
3. Kontrol geçerse:
 - SaveService.SaveAsync() – prestige öncesi güvenlik kaydı
 - OnPrestigeStarted event'i ateşlenir
 - ↳ EconomyService: CurrentResources = StartingGold
 - ↳ WaveSpawnManager: ResetForNewRun()
 - ↳ UpgradeApplicationSystem: ClearRunEffects()
 - ↳ (siz de burada UI animasyonunuzu tetikleyin)
4. PrestigeCount++
5. Yeni kalıcı çarpan hesaplanır (MultiplierFormula ile)
6. UpgradeApplicationSystem.SetPermanentMultiplier(newMult) çağrılır
7. OnPrestigeComplete(newCount, newMultiplier) event'i ateşlenir
8. SaveService.SaveAsync() – prestige sonrası kayıt

IPrestigeQuery Arayüzü

PrestigeStateManager, IPrestigeQuery arayüzünü uygular. Diğer sistemler doğrudan PrestigeStateManager'a bağımlı olmak yerine bu arayüzü kullanır:

```
// Namespace: EndlessEngine.Prestige
public interface IPrestigeQuery
{
    int PrestigeCount { get; }
    bool CanPrestige { get; }
    float GetPermanentMultiplier();
}

// Kullanım (bağımlılık enjeksiyonunda):
IPrestigeQuery query = _prestigeManager; // upcast
int count = query.PrestigeCount;
```

API

```
// Prestige tetikle (koşullar sağlanmıyorsa sessizce false döner)
bool ok = _prestigeManager.TryPrestige();

// Durum sorguları
int count = _prestigeManager.PrestigeCount;
bool canDo = _prestigeManager.CanPrestige; // MinWaveToPrestige sağlandı mı?
float mult = _prestigeManager.GetPermanentMultiplier();

// Olaylar
PrestigeStateManager.OnPrestigeStarted += () =>
{
    ShowPrestigeAnimation();
    // Burada prestige sırasında sıfırlanmasını istediğiniz özel sistemleri temizleyin
};

PrestigeStateManager.OnPrestigeComplete += (count, mult) =>
{
    ShowRewardScreen(count, mult);
    _skillTree.AddPoints(1); // her prestige'de 1 skill puanı ver
};

PrestigeStateManager.OnRealmUnlocked += (slug) =>
    UnlockRealmUI(slug); // belirli prestige sayısında yeni realm açılır
```

Sık Yapılan Hatalar

```
// HATA 1: SetCurrentWave çağrılmıyor → CanPrestige daima false
// Çözüm: WaveSpawnManager.OnWaveStarted'a abone olun (kurulum adım 3)

// HATA 2: ClearRunEffects çağrılmıyor → prestige sonrası eski efektler birikir
// Çözüm: OnPrestigeStarted'da çağırın:
PrestigeStateManager.OnPrestigeStarted += () =>
    UpgradeApplicationSystem.ClearRunEffects();

// HATA 3: RegisterServiceProvider eksik → PrestigeCount kayıt/yüklemede sıfırlanır
// Çözüm: _saveService.RegisterServiceProvider(_prestigeManager);
```

19. AscensionStateManager

Ne yapar: "Derin sıfırlama" — prestige sayacının da sıfırlandığı meta katman. Birden fazla Ascension Layer destekler. Her layer kendi `RequiredPrestigeCount` koşuluna ve `PermanentMultiplierPerAscension` değerine sahiptir. Tüm katmanların çarpanı `CascadeMultiplier` olarak birleşir ve tüm sistemi etkiler.

Prestige ile Ascension Farkı

	Prestige	Ascension
Ne sıfırlar	Altın, dalga, run efektleri	Bunlara ek olarak PrestigeCount
Ne kazandırır	Kalıcı gelir çarpanı	Cascade çarpanı (çok daha büyük)
Koşul	MinWaveToPrestige	RequiredPrestigeCount
Ne zaman	Her run sonunda mümkün	Nadir, milestone olayı

Config: AscensionDatabaseSO + AscensionLayerConfigSO

Create → Endless Engine → Ascension Database (tek bir veritabanı)
Create → Endless Engine → Ascension Layer Config (her katman için)

AscensionDatabaseSO, tüm AscensionLayerConfigSO referanslarını tutan kapsayıcıdır. Her layer ayrı bir SO'dur; veritabanına Inspector'dan eklenir.

Alan (AscensionLayerConfigSO)	Açıklama	Örnek
LayerIndex	Katman numarası (0'dan başlar)	0
RequiredPrestigeCount	Bu katmana girebilmek için gereken prestige sayısı	10
PermanentMultiplierPerAscension	Her ascension'da tüm gelire uygulanan çarpan	2.0

Başlatma

```
// AscensionStateManager'ın Initialize() metodu VARDIR — tüm bağımlılıkları alır:
_ascensionManager.Initialize(
    database: _ascensionDatabase, // AscensionDatabaseSO — tüm layer config'leri
    prestigeManager: _prestigeManager, // IPrestigeQuery — PrestigeCount okumak için
    saveService: _saveService, // ISaveNotifier — ascension sonrası kayıt
    economyService: _economyService, // Altın sıfırlama için
    generatorSystem: _generatorSystem // opsiyonel — generator sıfırlama için
);
_saveService.RegisterStateProvider(_ascensionManager);

// SaveProviderOrder.Ascension = 25 (Prestige = 30'dan ÖNCE çalışır — doğru sıra)
```

Cascade Çarpanı Nasıl Hesaplanır?

Birden fazla layer varsa cascade, her layer'ın kendi çarpanlarının çarpımıdır:

```

cascade = Π (layer_i.PermanentMultiplierPerAscension ^ count_i)

// Örnek: Layer0'da 2 ascension (2.0x), Layer1'de 1 ascension (3.0x):
// cascade = 2.0^2 × 3.0^1 = 4.0 × 3.0 = 12.0x

```

Ascension Akışı

```

1. TryTrigger(layerIndex, currentWaveNumber) çağrılır
2. CanTrigger kontrolü:
   → PrestigeCount >= layer.RequiredPrestigeCount mi?
   → Zaten bu layer'da ascension işlemde mi? (re-entrant guard)
3. Kontrol geçerse:
   → OnAscensionStarted(layerIndex) event'i
      ↳ EconomyService: altın sıfırlanır
      ↳ PrestigeStateManager: PrestigeCount = 0 (sıfırlanır!)
      ↳ GeneratorSystem: opsiyonel sıfırlama
   → Layer için ascension sayacı artar
   → Yeni cascade çarpanı hesaplanır
   → OnAscensionComplete(layerIndex, count, newCascade) event'i
   → OnAscensionResetRequested event'i – UI sıfırlama sinyali
   → SaveService.SaveAsync()

```

API

```

// Ascend etmeyi dene
bool ok = _ascensionManager.TryTrigger(layerIndex: 0, currentWaveNumber: _waveManager.CurrentWaveNumber);

// Kaç kez ascend edildi (layer bazlı)
int countL0 = _ascensionManager.GetLayer0Count();
int countL1 = _ascensionManager.GetCount(layerIndex: 1);

// Tüm katmanların birleşik çarpanı
float cascade = _ascensionManager.GetCascadeMultiplier();

// Koşul sağlanıyor mu?
bool canAscend = _ascensionManager.CanTrigger(layerIndex: 0, _waveManager.CurrentWaveNumber);

// Olaylar
AscensionStateManager.OnAscensionStarted += (layer) =>
    ShowAscensionAnimation(layer);

AscensionStateManager.OnAscensionComplete += (layer, count, cascade) =>
{
    cascadeText.text = $"Cascade: x{cascade:F1}";
    Debug.Log($"Ascension L{layer} #{count} | Yeni Cascade: {cascade:F2}x");
};

AscensionStateManager.OnAscensionResetRequested += () =>
{
    // PrestigeCount sıfırlandı – prestige UI'ını güncelle
    ResetPrestigeUI();
    ResetRunUI();
};

```

Sık Yapılan Hatalar

```

// HATA 1: PrestigeCount >= RequiredPrestigeCount olduğunu sanmak ama
//         SetCurrentWave çağrılmadığı için Prestige hiç yapılamamış
// Çözüm: PrestigeStateManager kurulumunda WaveSpawnManager.OnWaveStarted bağlantısını kontrol edin

// HATA 2: OnAscensionResetRequested'e abone olmadan UI'ı güncellemek
// Prestige sayacı sıfırlandığında UI'ın eski değeri göstermesini önleyin:
AscensionStateManager.OnAscensionResetRequested += () =>
    prestigeCountText.text = "0";

```


BÖLÜM F

Savaş Sistemi

20. WaveSpawnManager

21. AutoBattleController

22. DamageSystem

23. InputProvider

20. WaveSpawnManager

Ne yapar: Düşman dalgalarını yönetir. Her dalga düşmanları spawn eder; tamamlandığında sonraki dalgaya geçer. Dalga numarasını kayıt sistemine bildirir; belirli aralıklarda upgrade kart ekranını tetikler.

WaveConfigSO Neden Initialize'a Geçirilmez?

Eski versiyonda (v1.0.x) `Initialize(enemyManager, config: ConfigRegistry.Wave)` çağrısında config doğrudan geçiriliyordu. v1.1.0'da `WaveSpawnManager.Awake()` içinde `ConfigRegistry.Wave` 'e doğrudan erişir — `ConfigLoadingService` 'in `OnConfigsLoaded` event'i tetiklendikten sonra `Awake()` çalışacağı garanti edilir (`[DefaultExecutionOrder(-1000)]`). Bu yüzden Initialize parametresinden çıkarıldı.

Önemli: Bu değişiklik sadece `WaveConfigSO` 'yu etkiler. `EnemyManager` ve `SaveNotifier` hâlâ Initialize'a geçirilmelidir — bunlar sahnede birden fazla instance'ı olabilecek `MonoBehaviour`'lar olduğu için otomatik çözülmez.

Config: WaveConfigSO

Create → Endless Engine → Wave Config

Alan	Açıklama	Örnek
<code>TotalWavesPerRun</code>	Run başına toplam dalga sayısı	30
<code>WaveTransitionDelaySeconds</code>	İki dalga arası bekleme süresi	2.0
<code>UpgradeSelectionWaveInterval</code>	Kaç dalgada bir upgrade kart ekranı çıkar	5
<code>EnemiesPerWave</code>	Dalga→düşman sayısı eğrisi (AnimationCurve)	—
<code>EnemyScalingFactor</code>	Dalga numarasına göre HP/hasar artış çarpanı	1.12

Başlatma

```
// WaveConfigSO Initialize'a GEÇİRİLMEZ – Awake()'te ConfigRegistry.Wave'den okunur.
_waveManager.Initialize(
    enemyManager: _enemyManager,           // EnemyManager – spawn ve öldürme döngüsü
    saveNotifier: _saveService,             // IWaveSaveNotifier – dalga numarası kaydı
    healthSystem: _healthSystem             // opsiyonel – oyuncu ölümünde wave durdurma
);
_saveService.RegisterStateProvider(_waveManager);

// SaveProviderOrder.WaveAndCombat = 40
```

Dalga Döngüsü

```
StartFirstWave() çağrılır
→ WaveState: Idle → Spawning
→ OnWaveStarted(waveNumber) ateşlenir
  ↳ PrestigeStateManager.SetCurrentWave(wave) ← BUNU BAĞLAMAYI UNUTMAYIN
→ Düşmanlar spawn edilir (EnemiesPerWave animasyon eğrisinden)
→ Son düşman ölünce WaveState: Spawning → Transitioning
→ OnWaveComplete(waveNumber) ateşlenir
→ wave % UpgradeSelectionWaveInterval == 0 ise:
  ↳ OnUpgradeSelectionTriggered ateşlenir → siz savaşı durdurup kart ekranı gösterirsiniz
→ WaveTransitionDelaySeconds bekler
→ Sonraki dalgaya geçer
```

API

```
// İlk dalgayı başlat (Bootstrap sonunda veya "Başla" butonunda)
_waveManager.StartFirstWave();

// Mevcut dalga
int wave = _waveManager.CurrentWaveNumber;

// Durum makinesi
WaveState state = _waveManager.State; // Idle | Spawning | Transitioning

// Prestige/Ascension sonrası run sıfırlama
_waveManager.ResetForNewRun();

// Olaylar
WaveSpawnManager.OnWaveStarted += (wave) =>
{
    waveText.text = $"Dalga {wave}";
    _prestigeManager.SetCurrentWave(wave); // ← ZORUNLU BAĞLANTI
};

WaveSpawnManager.OnWaveComplete += (wave) =>
    Debug.Log($"Dalga {wave} tamamlandı");

WaveSpawnManager.OnUpgradeSelectionTriggered += () =>
{
    _abc.StopCombat(); // savaşı durdur
    ShowUpgradeCardScreen();
    // Kart seçilince: _abc.StartCombat() veya _abc.NotifyUpgradeSelected()
};
```

Yaygın Hata: PrestigeStateManager Bağlantısı

WaveSpawnManager.OnWaveStarted → _prestigeManager.SetCurrentWave(wave) bağlantısı kurulmazsa PrestigeManager.CanPrestige her zaman false döner. Bu, prestige butonunun hiç aktif olmadığı bir hata olarak görünür ama gerçek neden wave numarasının hiç bildirilmemesidir.

21. AutoBattleController

Ne yapar: Otomatik savaş döngüsü. En yakın düşmanı hedef alır, saldırı aralığına göre hasar verir.

Config: PlayerBaseStatConfigSO

Create → Endless Engine → Player Base Stat Config

Alan	Açıklama
BaseAttackDamage	Temel hasar
BaseAttackInterval	Saldırı aralığı (saniye)
BaseMaxHP	Maksimum can
BaseCritChance	Kritik şans (0-1)
BaseCritMultiplier	Kritik çarpan

Neden BaseStatUpgradeProvider Gerekliyor?

Eski versiyonda (v1.0.x) şöyle kullanılıyordu:

```
// ESKİ (v1.0.x) – ARTIK GEÇERSİZ:
_abc.Initialize(..., statProvider: _upgradeApplicationSystem, ...);
```

UpgradeApplicationSystem static bir class olduğu için instance referansı _upgradeApplicationSystem diye bir şey olmaz. v1.1.0'da IUpgradeStatProvider arayüzü oluşturuldu ve BaseStatUpgradeProvider bu arayüzü uygulayan somut sınıf oldu:

```
// IUpgradeStatProvider – EndlessEngine.Combat namespace'inde
public interface IUpgradeStatProvider
{
    float GetAttackDamage();
    float GetAttackInterval();
    float GetCritChance();
    float GetCritMultiplier();
    float GetMoveSpeed();
}
```

BaseStatUpgradeProvider(PlayerBaseStatConfigSO config) bu metodları UpgradeApplicationSystem.GetEffectiveStat(StatType.X) üzerinden hesaplar. Böylece AutoBattleController stat sistemine arayüz üzerinden bağımlıdır — static class'a doğrudan değil.

Başlatma

```
// ADIM 1: IUpgradeStatProvider oluştur
// Namespace: EndlessEngine.Combat
var statProvider = new BaseStatUpgradeProvider(ConfigRegistry.Player);
// ConfigRegistry.Player = PlayerBaseStatConfigSO (base değerler burada)
// Actual saldırı hesabı: BaseAttackDamage × UpgradeApplicationSystem.GetEffectiveStat(Damage)

// ADIM 2: Initialize
_abc.Initialize(
    enemyManager: _enemyManager,
    waveSpawnManager: _waveSpawnManager,
    statProvider: statProvider, // IUpgradeStatProvider (yukarıda oluşturduğumuz)
    playerConfig: ConfigRegistry.Player, // PlayerBaseStatConfigSO
    waveConfig: ConfigRegistry.Wave, // ConfigRegistry'den otomatik alınır
    playerId: 1 // DamageSystem'da oyuncuyu tanımlar
);

// ADIM 3: PlayerHealthComponent bağlantısı (hasar almak için)
_abc.SetPlayerQuery(_playerHealthComponent); // IPlayerQuery
// PlayerHealthComponent IPlayerQuery uygulamalar: IsInIdleRecovery, Position

// ADIM 4: Savaşı başlat – Initialize'dan HEMEN SONRA çağrılmalı
_abc.StartCombat();
// StartCombat() çağrılmadan düşmanlar spawn edilse de saldırı gerçekleşmez
```

API

```
// Mevcut durum
CombatState state = _abc.State; // Idle / Fighting / WaveTransition / UpgradeSelection

// Upgrade seçildikten sonra savaşa devam et
_abc.NotifyUpgradeSelected();

// Olaylar
AutoBattleController.OnEnemyKilled += (agent) => AddGold(agent.GoldDrop);
AutoBattleController.OnWaveComplete += (wave) => OnWaveCleared(wave);
AutoBattleController.OnPlayerDied += () => ShowGameOverScreen();
AutoBattleController.OnUpgradeSelectionTriggered += () => ShowUpgradeCards();
```

22. DamageSystem

Ne yapar: Tüm hasar olaylarının işlendiği bus. Kritik hesabı, zırh azaltma, hasar türü ayrımı.

Önemli: Bu da `static class` 'tır — instance veya `Initialize()` gerekmez. Otomatik çalışır; sadece eventlerine abone olun.

API

```
// Namespace: EndlessEngine.Damage
using EndlessEngine.Damage;

// Hasar gönder (genellikle AutoBattleController çağırır – direkt çağırmanız gerekmez)
DamageSystem.ResolveDamage(
    rawDamage:      10f,
    attacker:        AttackerType.Player,
    damageType:      DamageType.Physical,
    targetId:        enemyId,
    hitPos:          transform.position,
    isPlayerInvincible: false
);

// Çözümlenen hasarı dinle (VFX, ses, UI için)
DamageSystem.OnDamageResolved += (hit) =>
{
    // hit.TargetId      – hedef entity ID'si
    // hit.FinalDamage   – uygulanacak son hasar miktarı
    // hit.IsCritical    – kritik vuruş muydu?
    // hit.HitPosition   – dünya pozisyonu
    // hit.AttackerType  – Player / Enemy
    SpawnDamageNumber(hit.HitPosition, hit.FinalDamage, hit.IsCritical);
};

// Engellenen hasarı dinle (kalkan/zırh tampon vb.)
DamageSystem.OnDamageBlocked += (hit) =>
{
    PlayBlockEffect(hit.HitPosition);
};
```

23. InputProvider

Ne yapar: Unity Input System'ı soyutlar. Tüm oyun kodu `IInputProvider` arayüzü üzerinden giriş okur.

IInputProvider Arayüzü

```
public interface IInputProvider
{
    Vector2 GetMoveVector();           // WASD / analog
    bool GetConfirmPressed();          // Space / A
    bool GetCancelPressed();           // Esc / B
    bool GetPausePressed();            // Esc / Start
    Vector2 GetMouseWorldPosition();    // fare dünya pozisyonu
    bool GetPointerClickedThisFrame(); // sol tık (bu frame)
    event Action OnPausePressed;        // debounce'lu pause eventi
}
```

Kurulum

1. Player GameObject'ine `PlayerInput` + `InputProviderUnity` ekleyin.
2. `GameInputActions.inputactions` asset'ini `PlayerInput`'a atayın.
3. Bootstrap'te `_inputProvider = playerGO.GetComponent<InputProviderUnity>()`.

Kural: Tek Dosya

`InputProviderUnity.cs` dışındaki hiçbir dosya `UnityEngine.InputSystem` namespace'ini import etmez. Oyun kodu sadece `IInputProvider` kullanır.

BÖLÜM G

İçerik Sistemleri

24. BuildingService

25. PetService

26. EventService

27. ChallengeService

28. MilestoneTracker

29. QuestService

30. MinigameService

31. MergeService

32. ConversionService

33. TraitService

34. UnlockLogService

24. BuildingService

Ne yapar: Grid tabanlı bina yerleştirme. Binalar belirli hücelere yerleştirilir, her biri pasif bonus sağlar.

Config: BuildingConfigSO

Create → Endless Engine → Building Config

Alan	Açıklama
BuildingId	Benzersiz ID
GridSize	Kapladığı hücre boyutu (Vector2Int)
BuildCost	Yapım maliyeti
UpgradeCosts[]	Tier başına yükseltme maliyeti
ProductionBonus	Sağladığı gelir bonusu

Kullanım

```
var result = _buildingService.TryPlace("farm", gridX: 2, gridY: 3);
if (result.Success)
    gridView.Render(result.Instance);

bool upgraded = _buildingService.TryUpgrade(instanceId);
_buildingService.Remove(instanceId);

BuildingService.OnBuildingPlaced += (inst) => UpdateGrid();
BuildingService.OnBuildingUpgraded += (inst) => UpdateGrid();
BuildingService.OnBuildingRemoved += (id) => UpdateGrid();
BuildingService.OnBuildingProduced += (id, amount) => ShowProduction(amount);
```

25. PetService

Ne yapar: Yoldaş sistemi. Her pet pasif bonus ve aktif yetenek sağlar; kilitleme, seviye atlatma, evrimleşme destekler.

Config: PetConfigSO

Create → Endless Engine → Pet Config

Alan	Açıklama
PetId	Benzersiz ID
DisplayName	Gösterim adı
AffectedStat	Etkilediği stat
BaseEffect	Temel etki değeri
LevelUpCosts[]	Seviye başına maliyet
EvolveAtLevel	Evrin için gereken seviye
EvolvesToPetId	Evrin sonucu pet ID'si

Başlatma

```
_petService.Initialize(
    configs: ConfigRegistry.PetConfigs,
    economy: _economyService
);
_saveService.RegisterStateProvider(_petService);
```

API

```
bool ok = _petService.TryEquip("fire-cat");
bool lvl = _petService.TryLevelUp("fire-cat");
bool evo = _petService.TryEvolve("fire-cat");
int level = _petService.GetLevel("fire-cat");

PetService.OnPetEquipped += (cfg) => ShowPetIcon(cfg);
PetService.OnPetLeveledUp += (cfg, level) => ShowLevelUp(level);
PetService.OnPetEvolved += (from, to) => ShowEvolution(to);
```

26. EventService

Ne yapar: Takvim tabanlı özel etkinlikler — belirli zaman dilimlerinde bonus çarpanlar, özel düşmanlar, sınırlı içerik aktive edilir.

Config: EventScheduleConfigSO

Create → Endless Engine → Event Schedule Config

Alan	Açıklama
EventId	Benzersiz ID
DisplayName	Gösterim adı
StartMonth , StartDay	Başlangıç tarihi
EndMonth , EndDay	Bitiş tarihi
IncomeMultiplier	Bu etkinlik süresince gelir çarpanı
ResearchMultiplier	Bu etkinlik süresince araştırma hızı

Başlatma

```
_eventService.Initialize(
    events: new EventScheduleConfigSO[] { _summerEvent, _xmasEvent }
);
```

API

```
// Aktif etkinlikleri kontrol et
IEnumerable<EventScheduleConfigSO> active = _eventService.GetActiveEvents();

// Belirli bir etkinlik aktif mi?
bool isActive = _eventService.IsActive("summer-fest");

// Birleşik gelir çarpanı (tüm aktif etkinlikler)
float incomeBonus = _eventService.GetCombinedIncomeMultiplier();

// Takvim kontrolünü tetikle (genellikle TickEngine'e bağlanır)
_eventService.CheckSchedule();
_tickEngine.OnTick += (_) => _eventService.CheckSchedule();

// Olaylar
EventService.OnEventActivated += (ev) => ShowEventBanner(ev.DisplayName);
EventService.OnEventDeactivated += (ev) => HideEventBanner();
```

27. ChallengeService

Ne yapar: Opsiyonel kısıtlama modları. "Sadece 1 jeneratör", "saldırı yok ama 3x ödül" gibi zorluklar oyuna derinlik katar.

Config: ChallengeConfigSO

Alan	Açıklama
ChallengeId	Benzersiz ID
DisplayName	Görüntülenecek ad
Modifiers[]	Uygulanan kısıtlamalar (ChallengeModifier)
RewardMultiplier	Tamamlama ödül çarpanı
VictoryWave	Hedef dalga sayısı

Başlatma

```
_challengeService.Initialize(
    economyService: _economyService,
    upgradeTree: _upgradeTree
);
```

API

```
_challengeService.ActivateChallenge(config);
_challengeService.CancelChallenge();

bool active = _challengeService.IsRunActive;
bool disabled = _challengeService.IsSystemDisabled("auto-battle");
float reward = _challengeService.ActiveChallenge?.RewardMultiplier ?? 1f;

ChallengeService.OnChallengeCompleted += (rewardGold) => ShowReward(rewardGold);
ChallengeService.OnChallengeFailed += () => ShowFailScreen();
```

28. MilestoneTracker

Ne yapar: Belirli eşiklere ulaşıncı ödül ve bildirim tetikler ("İlk 1000 altın", "10. prestige" vb.).

Config: MilestoneConfigSO

Alan	Açıklama
MilestoneId	Benzersiz ID
TrackedStat	Hangi değer izleniyor
Threshold	Eşik değeri
RewardType	Ödül türü
RewardValue	Ödül miktarı

Başlatma

```
_milestoneTracker.Initialize(
    database: _milestoneDatabase,
    economyService: _economyService,
    prestigeManager: _prestigeManager, // opsiyonel
    currencyService: _currencyService, // opsiyonel
    generatorSystem: _generatorSystem // opsiyonel
);
_saveService.RegisterStateProvider(_milestoneTracker);
```

API

```
MilestoneTracker.OnMilestoneCompleted += (cfg) =>
    ShowPopup($"Kilometre taşı: {cfg.DisplayName}!");
```

29. QuestService

Ne yapar: Kısa vadeli görevler — "100 düşman öldür", "3 jeneratör al" gibi günlük/haftalık hedefler.

Config: QuestConfigSO

Create → Endless Engine → Quest Config

Alan	Açıklama
QuestId	Benzersiz ID
DisplayName	Gösterim adı
Conditions[]	IQuestCondition listesi (ResourceThresholdCondition, WaveReachedCondition)
ResetType	Never / Daily / Weekly
RewardGold	Tamamlama ödülü

Başlatma

```
_questService.Initialize(
    configs: new QuestConfigSO[] { _dailyQuest1, _dailyQuest2 },
    economy: _economyService,
    currency: null // opsiyonel ikincil para birimi
);
_saveService.RegisterStateProvider(_questService);

// Özel koşullar ekle
_questService.RegisterCondition(new WaveReachedCondition(_waveManager));
```

API

```
// Görev tamamlandı mı?
bool done = _questService.IsCompleted("kill-100-enemies");

// İlerleme (0.0 → 1.0)
float progress = _questService.GetProgress("kill-100-enemies");

// Manuel kontrol tetikle (koşullar değişince)
_questService.Check();

// Olaylar
QuestService.OnQuestCompleted += (cfg) => GiveReward(cfg.RewardGold);
QuestService.OnQuestReset += (cfg) => RefreshQuestUI();
```

30. MinigameService

Ne yapar: Belirli aralıklarla tetiklenen aktif mini oyunlar ("aktif beceri" sistemi). Her mini oyun cooldown'a sahiptir; tetiklendiğinde oyuncudan eylem kaydedilir, süre dolunca ödül hesaplanır.

Config: ActiveSkillConfigSO

Create → Endless Engine → Active Skill Config

Alan	Açıklama
SkillId	Benzersiz ID
DisplayName	Gösterim adı
CooldownSeconds	Yeniden tetiklenme süresi
SessionDurationSeconds	Mini oyun süresi
BaseReward	Temel ödül miktarı
ActionsPerRewardTier	Tier atlamak için gereken eylem sayısı

Başlatma

```
_minigameService.Initialize(
    skills: new ActiveSkillConfigSO[] { _tapFrenzySkill },
    economy: _economyService
);
```

API

```
// Mini oyun kullanılabilir mi? (cooldown bitti mi?)
bool canTrigger = _minigameService.CanTrigger("tap-frenzy");

// Tetikle
bool started = _minigameService.TryTrigger("tap-frenzy");

// Oyun içinde eylem kaydet (ör. her tıklamada)
if (_minigameService.IsSessionActive)
    _minigameService.RecordAction();

// Oturumu erken bitir
_minigameService.EndSession();

// Kalan cooldown süresi (UI için)
float remaining = _minigameService.GetCooldown("tap-frenzy");

// Olaylar
MinigameService.OnMinigameStarted += (skill) => ShowMinigameOverlay(skill.DisplayName);
MinigameService.OnActionRecorded += (skill, count) => UpdateActionCounter(count);
MinigameService.OnMinigameEnded += (skill, reward) => ShowReward(reward);
MinigameService.OnSkillReady += (skillId) => ShowReadyIndicator(skillId);
```

31. MergeService

Ne yapar: Merge (birleştirme) mekaniği — 2 aynı tier'daki nesneyi birleştir, bir üst tier nesne elde et.

Config: MergeConfigSO

Create → Endless Engine → Merge Config

Her zincir `MergeConfigSO` içinde tanımlanır; `InputItem` (girdi tier) → `OutputItem` (çıktı tier) eşleşmelerini içerir.

Başlatma

```
_mergeService.Initialize(
    configs:    new MergeConfigSO[] { _gemMergeChain },
    inventory:  _inventoryService,
    economy:    _economyService    // opsiyonel
);
```

API

```
// Birleştirilebilir mi kontrol et
bool canMerge = _mergeService.CanMerge(itemConfig);

// Birleştir
MergeResult result = _mergeService.TryMerge(itemConfig);

if (result.Success)
{
    // result.ResultItem – elde edilen üst tier config
    // result.ConsumedCount – tüketilen öge sayısı
    // result.ResultTier – yeni tier seviyesi
    gridView.Remove(selectedSlotA);
    gridView.Remove(selectedSlotB);
    gridView.Place(result.ResultItem, targetPosition);
}

// Olaylar
MergeService.OnMergeCompleted += (itemId, tier, result) =>
{
    PlayMergeVFX(gridView.GetPosition(itemId));
    ShowTierUpText(result.ResultTier);
};
MergeService.OnMergeFailed += (itemId, tier, reason) =>
    ShowMergeError(reason);
```

32. ConversionService

Ne yapar: Bir kaynak türünü diğerine dönüştürür (altın → kristal, jem → enerji vb.). Cooldown desteği vardır.

Config: ConversionDatabaseSO

Create → Endless Engine → Conversion Database

Her tarif `ConversionRecipeSO` içinde tanımlanır: `RecipeId`, `InputCurrency`, `InputAmount`, `OutputCurrency`, `OutputAmount`, `CooldownSeconds`.

Başlatma

```
_conversionService.Initialize(
    database:      _conversionDatabase,
    economy:       _economyService,
    currencyService: _currencyService    // opsiyonel
);
```

API

```
// Dönüştür (1 kez)
bool ok = _conversionService.TryConvert("gold-to-gems");

// Toplu dönüştür
bool ok = _conversionService.TryConvert("gold-to-gems", count: 10);

// Cooldown bilgisi (UI için)
bool onCooldown = _conversionService.IsOnCooldown("gold-to-gems");
float remaining = _conversionService.GetCooldownRemaining("gold-to-gems");

// Tarifi al (UI'da gösterim için)
ConversionRecipeSO recipe = _conversionService.GetRecipe("gold-to-gems");

// Olaylar
ConversionService.OnConverted += (recipeId, count, inputSpent, outputGained) =>
    ShowConversionResult(outputGained);
ConversionService.OnConversionFailed += (recipeId, reason) =>
    ShowError($"Dönüştürülemedi: {reason}");
```

33. TraitService

Ne yapar: Oyuncunun kalıcı nitelik seçimi. Prestige/Ascension'da sunulan trait'ler oyun stilini şekillendirir.

Config: TraitConfigSO

Create → Endless Engine → Trait Config

Alan	Açıklama
TraitId	Benzersiz ID
DisplayName	Gösterim adı
Effects[]	SkillEffect listesi (AffectedStat, EffectValue)

Başlatma

```
_traitService.Initialize(
    allTraits: _allTraitConfigs, // TraitConfigSO[]
    prestige: _prestigeManager,
    saveService: _saveService
);
_saveService.RegisterStateProvider(_traitService);
```

API

```
// Seçim ekranında sunulanlar
ICollection<string> chosen = _traitService.ChosenTraitIds;

// Seçim bekliyor mu?
bool pending = _traitService.HasPendingSelection;

// Trait seç (seçim ekranı butonu)
bool ok = _traitService.ChooseTrait("speed-demon");

// Belirli trait seçildi mi?
bool has = _traitService.IsChosen("speed-demon");

// Bu stat için toplam modifier (UpgradeApplicationSystem ile entegrasyon)
Modifier mod = _traitService.GetModifier(StatType.AttackInterval);

// Olaylar
TraitService.OnTraitSelectionAvailable += (options) =>
    ShowTraitSelectionScreen(options); // options: TraitConfigSO[]
TraitService.OnTraitChosen += (trait) =>
    ShowChosenTraitAnimation(trait.DisplayName);
```

34. UnlockLogService

Ne yapar: Kilidi açılan içeriklerin kayıtlı geçmişini tutar. Hangi realm'lerin, başarımların, içeriklerin açıldığını loglar ve UI'da gösterim sağlar.

Config: UnlockEntryConfigSO

Create → Endless Engine → Unlock Entry Config

Alan	Açıklama
EntryId	Benzersiz ID
DisplayName	Gösterim adı
Category	Realm / Achievement / Feature / Item
IsVisible	Unlock log UI'da görünsün mü?

Başlatma

```
_unlockLogService.Initialize(
    entries: new UnlockEntryConfigSO[] { _realmEntry, _achievementEntry }
);
_saveService.RegisterStateProvider(_unlockLogService);
```

API

```
// Kilidi aç (önceden tanımlanmış entry)
_unlockLogService.Unlock("fire-realm");

// Dinamik kilit açma (SO olmayan geçici kayıtlar)
_unlockLogService.UnlockDynamic("custom-achievement-id");

// Açık mı?
bool unlocked = _unlockLogService.IsUnlocked("fire-realm");

// Toplam açılan sayısı
int total = _unlockLogService.TotalUnlocked;

// Kategoriye göre listele
IReadOnlyList<UnlockEntryConfigSO> realms = _unlockLogService.GetUnlocked(UnlockCategory.Realm);

// Tüm görünür girişler (UI için)
IReadOnlyList<(UnlockEntryConfigSO Config, bool IsUnlocked)> visible = _unlockLogService.GetVisible();

// Olay
UnlockLogService.OnEntryUnlocked += (entry) =>
    ShowUnlockAnimation(entry.DisplayName);
```

BÖLÜM H

Destek Sistemleri

35. StatisticsService

36. NotificationService

37. LeaderboardService

38. TutorialService

39. RealmSystem

40. AudioService

41. BigDouble

42. TimeBoostService

43. ZoneSystem

44. CursorYieldService

45. InventoryService

46. DropResolver

47. PlayerHealthComponent

48. GameFlowState ve RunSummaryData

49. IdleModule

35. StatisticsService

Ne yapar: Koşu ve ömür boyu istatistikleri toplar. "En yüksek dalga", "toplam tıklama", "kaç prestige yapıldı" vb.

Başlatma

```
_statisticsService.Initialize(ConfigRegistry.StatDefinitions);
_saveService.RegisterStateProvider(_statisticsService);
```

API

```
// Artır
_statisticsService.Add("harvest.total_gold", goldAmount);
_statisticsService.Add("clickloop.total_targets_destroyed", 1);

// En yüksek güncelle (azalmaz)
_statisticsService.SetIfHigher("highest_wave", currentWave);

// Oku
double total = _statisticsService.Get("harvest.total_gold");

// Sistem sabit ID'leri:
// HarvestLoopService.StatIdTotalGold       = "harvest.total_gold"
// HarvestLoopService.StatIdTotalNodes      = "harvest.total_nodes_harvested"
// HarvestLoopService.StatIdBestCombo       = "harvest.best_combo_multiplier"
// ClickLoopService.StatIdTotalGold         = "clickloop.total_gold"
// ClickLoopService.StatIdTotalDestroyed    = "clickloop.total_targets_destroyed"
// ClickLoopService.StatIdBestCombo         = "clickloop.best_combo_multiplier"
```

StatDefinitionSO

Create → Endless Engine → Stat Definition

Alan	Açıklama
StatId	Benzersiz string ID
DisplayName	Kullanıcıya gösterilen ad
IsPeakValue	true = azalmaz (yüksek dalga), false = birikir (toplam altın)

36. NotificationService

Ne yapar: Oyun içi bildirim kuyruğu. Tek seferde tek bildirim gösterir; yeniler kuyruğa eklenir.

Bu bir Singleton MonoBehaviour'dır. NotificationService.Instance üzerinden erişilir veya Inspector'dan referans alınır.

Config: NotificationConfigSO

Create → Endless Engine → Notification Config

Alan	Açıklama
Title	Bildirim başlığı
Body	Açıklama metni
Duration	Gösterim süresi (saniye)
Icon	İkon sprite

Kullanım

```
// Bildirim kuyruğuna ekle
NotificationService.Instance.Enqueue(_researchDoneConfig);

// Metin override ile gönder
NotificationService.Instance.Enqueue(_genericConfig, overrideText: "Prestige edildi!");

// Tüm kuyruğu temizle
NotificationService.Instance.Clear();

// Mevcut aktif bildirim
NotificationItem active = NotificationService.Instance.Active;

// Kaç bildirim bekliyor
int waiting = NotificationService.Instance.QueueCount;

// Olaylar
NotificationService.OnNotificationShown += (item) => PlayNotificationSound();
NotificationService.OnNotificationDismissed += (item) => OnNextNotification();
```

Örnek: Araştırma bitince bildir

```
ResearchService.OnNodeCompleted += (treeId, nodeId) =>
{
    NotificationService.Instance.Enqueue(
        _researchConfig,
        overrideText: $"Araştırma tamamlandı: {nodeId}"
    );
};
```

37. LeaderboardService

Ne yapar: Yerel skor tabloları (yerleşik). Steam entegrasyonu ile bulut liderlik tablosu da desteklenir.

Config: LeaderboardConfigSO

Create → Endless Engine → Leaderboard Config

Alan	Açıklama
BoardId	Benzersiz ID
DisplayName	Gösterim adı
MaxEntries	Tutulacak maksimum kayıt
SortOrder	Descending / Ascending

Başlatma

```
_leaderboardService.Initialize(
    configs: new LeaderboardConfigSO[] { _highWaveBoard, _totalGoldBoard }
);
```

API

```
// Skor gönder
bool isNew = _leaderboardService.SubmitScore("highest-wave", playerName, waveNumber);

// Tablodaki tüm kayıtlar
IReadOnlyList<LeaderboardEntry> entries = _leaderboardService.GetBoard("highest-wave");

// Oyuncunun sırası
int rank = _leaderboardService.GetRank("highest-wave", playerScore);

// Bu skor yeni rekor mu?
bool highScore = _leaderboardService.IsHighScore("highest-wave", playerScore);

// Olay
LeaderboardService.OnScoreSubmitted += (boardId, entry) =>
{
    if (_leaderboardService.IsHighScore(boardId, entry.Score))
        ShowNewHighScoreEffect();
    RefreshLeaderboardUI(boardId);
};
```

Steam Liderlik Tablosu

```
// Steam entegrasyonu için SteamLeaderboardService kullanın (ayrı component)
// SteamService başlatıldıktan sonra otomatik bağlanır.
```


38. TutorialService

Ne yapar: Adım adım öğretici — koşul tabanlı adımlar sırayla ilerler. Her adım event veya manuel tamamlama ile geçilir.

Config: TutorialStepConfigSO

Create → Endless Engine → Tutorial Step Config

Alan	Açıklama
StepId	Benzersiz ID
TriggerEvent	Bu event gerçekleşince adım başlar
CompletionEvent	Bu event gerçekleşince adım biter
HighlightTargetId	Gösterilecek UI öğesinin ID'si
DialogueText	Öğretici mesajı

Başlatma

```
_tutorialService.Initialize(
    steps: new TutorialStepConfigSO[] { _step1, _step2 },
    saveService: _saveService
);
_saveService.RegisterStateProvider(_tutorialService);

// Öğreticiyi başlat
_tutorialService.Begin();
```

API

```
// Öğretici aktif mi?
bool active = _tutorialService.IsActive;
bool finished = _tutorialService.IsFinished;

// Mevcut adım
TutorialStepConfigSO currentStep = _tutorialService.CurrentStep;

// Adımı tamamla (manuel tamamlama için)
_tutorialService.CompleteCurrentStep();

// Event ile ilerleme (TriggerEvent ile eşleşen adım otomatik ilerler)
_tutorialService.NotifyEvent("first-generator-bought");
_tutorialService.NotifyTap(); // dokunma/tıklama bildirimi
_tutorialService.NotifyHighlightTapped(); // vurgulanan öğeye dokunuldu

// Öğreticiyi atla
_tutorialService.Skip();

// Olaylar
TutorialService.OnStepStarted += (step) => ShowTutorialArrow(step.HighlightTargetId);
TutorialService.OnStepCompleted += (step) => HideTutorialArrow();
TutorialService.OnTutorialFinished += () => HideTutorialUI();
```

39. RealmSystem

Ne yapar: Farklı oyun bölgelerini yönetir. Her realm kendi config paketine sahiptir — düşman stats, ekonomi, görsel tema değişir.

Dikkat: RealmConfigSystem'in Initialize() metodu yoktur. Sahnede component olarak bulundurulması yeterlidir; OnEnable içinde ConfigRegistry'den otomatik okur.

Yeni Realm Oluşturma

Tools → Endless Engine → Content Pack Wizard → Realm adı gir → Create All

Bu işlem Assets/Configs/[RealmName]/ altında 9 SO + RealmPackSO oluşturur ve RealmRegistry'ye otomatik ekler.

Realm Değiştirme

```
// Realm değişimi (async)
await ConfigRegistry.BeginRealmSwapAsync(ConfigRegistry.RealmRegistry.GetPack("volcano"));

// Realm değişince tüm sistemleri güncelle
ConfigRegistry.OnRealmSwapped += () =>
{
    // Tüm ConfigRegistry property'leri artık yeni realm'e işaret eder
    _resourceHardCap = ConfigRegistry.Economy.ResourceHardCap;
    _upgradeTree.RebuildForPrestige(); // yükseltme ağacını yeniden yükle
};
```

Mevcut Realm'leri Listele

```
// Kullanıcıya realm seçim ekranı için
RealmDisplayData[] available = _realmSystem.GetAvailableRealms();
foreach (var realm in available)
    realmUI.AddButton(realm.DisplayName, realm.Slug);

// Realm seç
await _realmSystem.SelectRealmAsync("fire-realm");
```

40. AudioService

Ne yapar: Pool tabanlı SFX çalma, müzik yönetimi, AudioManager snapshot geçişleri.

API

```
_audioService.PlaySFX(clip, volume: 1f, pitch: 1f);  
_audioService.SetSFXVolume(0.8f); // PlayerPrefs'e kaydeder  
_audioService.SetMusicVolume(0.5f);  
_audioService.Duck(); // Müziği kısaltma (ör. dialog sırasında)  
_audioService.Unduck();
```

41. BigDouble

Ne yapar: `1e308`'i aşan büyük sayıları temsil eden özel sayı tipi. Standart `double` sınırını aşan idle ekonomiler için.

Ne Zaman Kullanılır?

Oyuncunun 100 saatte ulaşabileceği değer `~1.8e308`'i aşacaksa `BigDouble` gerekir.

Aktifleştirme

`EconomyConfigS0` → `NumberBackend` → `BigDouble`. Tüm `EconomyService` API'si otomatik olarak `BigDouble` kullanır — kod değişikliği gerekmez.

Yapısı

$\text{değer} = \text{Mantissa} \times 10^{\text{Exponent}}$

```
var bd = new BigDouble(1.5, 300); // 1.5 × 10^300
bd.Mantissa // 1.5
bd.Exponent // 300
bd.Format(BigDouble.Notation.Letter, 2) // "1.50aa"
```

Dikkat Edilecekler

- `BigDouble(0.0, 999)` → `IsZero = true`
- `ToDouble()` için `exponent > 308` → `Infinity` döner; sadece görüntüleme için kullanın
- Backend değiştirirseniz mevcut kayıtlar için `SaveMigration` yazın

42. TimeBoostService

Ne yapar: TickEngine'in TimeScale'ini geçici olarak artıran hız boost sistemi. "2x hız" gibi ücretli veya ödüllü hız güçlendirmeleri için kullanılır. Aynı anda yalnızca bir boost aktif olabilir; yeni aktivasyon mevcut boost'u iptal eder.

Config: TimeBoostConfigSO

Create → Endless Engine → Time Boost Config

Alan	Açıklama	Örnek
BoostId	Benzersiz ID	"boost-2x"
DisplayName	Gösterim adı	"2x Hız"
TimeScaleMultiplier	Uygulanacak zaman çarpanı	2.0
DurationSeconds	Boost süresi	30
GoldCost	Ücretli aktivasyon maliyeti (0 = ücretsiz)	500

Başlatma

```
_timeBoostService.Initialize(
    tickEngine: _tickEngine,
    economyService: _economyService // null olabilir – ücretli boost kullanmıyorsanız
);
```

API

```
// Ücretsiz aktivasyon (reklam ödülü, quest ödülü vb.)
_timeBoostService.TryActivate(_boost2xConfig);

// Ücretli aktivasyon (altın düşürür, başarısızsa false döner)
bool ok = _timeBoostService.TryActivatePaid(_boost2xConfig);

// Mevcut durum
bool active = _timeBoostService.IsActive;
float remaining = _timeBoostService.RemainingSeconds;
TimeBoostConfigSO cfg = _timeBoostService.ActiveConfig; // null ise aktif yok

// İptal et
_timeBoostService.Cancel();

// Olaylar
TimeBoostService.OnBoostStarted += (config, remaining) =>
    boostUI.Show($"{config.DisplayName} – {remaining:F0}s");

TimeBoostService.OnBoostTick += (remaining) =>
    boostTimer.text = $"{remaining:F0}s";

TimeBoostService.OnBoostEnded += () =>
    boostUI.Hide();
```

UI Countdown Örneği

```
void Start()
{
    TimeBoostService.OnBoostStarted += (cfg, rem) =>
    {
        boostPanel.SetActive(true);
        boostLabel.text = cfg.DisplayName;
    };
    TimeBoostService.OnBoostTick += (rem) =>
        boostTimerText.text = $"{Mathf.CeilToInt(rem)}s";
    TimeBoostService.OnBoostEnded += () =>
        boostPanel.SetActive(false);
}
```

Dikkat

- Boost sırasında `TickEngine.TimeScale` doğrudan değiştirilir; boost bitince eski değere döner.
- Birden fazla boost aynı anda **aktif olamaz** — yeni `TryActivate` çağrısı öncekini iptal eder.

43. ZoneSystem

Ne yapar: Dünyada tanımlı bölgeleri (zone) yönetir. Her zone altın üretir; cursor/oyuncu içine girince hover çarpanı devreye girer. Pasif mod veya aktif (cursor-içinde) mod desteği vardır. Prestij geçidi ve yükseltme sistemi içerir.

Config: ZoneConfigSO

Create → Endless Engine → Zone Config

Alan	Açıklama	Örnek
ZoneId	Benzersiz ID	"forest-zone"
DisplayName	Gösterim adı	"Orman"
BaseYieldPerSecond	Saniyede temel altın üretimi	5.0
UnlockCost	Kilidi açma maliyeti	500
UpgradeCostBase	İlk yükseltme maliyeti	200
UpgradeCostMultiplier	Yükseltme başına maliyet çarpanı	1.5
HoverMultiplier	Cursor içindeyken uygulanan gelir çarpanı	3.0
PassiveMode	true = her tick üretir; false = sadece cursor içindeyken	true
PrestigeGateRequired	Erişim için gereken prestige sayısı	0

Başlatma

```
_zoneSystem.Initialize(
    configs:    ConfigRegistry.ZoneConfigs, // ZoneConfigSO[]
    economy:    _economyService,
    gameFlow:   _gameFlowStateMachine,
    input:      _inputProvider,
    saveNotifier: _saveService
);

// Prestige sayısını zone sistemine bildir (kapılı zone'lar için)
_zoneSystem.SetPrestigeCountGetter(() => _prestigeManager.PrestigeCount);

_saveService.RegisterStateProvider(_zoneSystem);
```

API

```
// Kilidi aç
bool ok = _zoneSystem.TryUnlock("forest-zone");

// Yükselt
bool upgraded = _zoneSystem.TryUpgrade("forest-zone");

// Bir sonraki yükseltme maliyeti
long cost = _zoneSystem.GetUpgradeCost("forest-zone");

// Cursor zone içinde mi? (fizik sorgusu)
bool inside = _zoneSystem.IsCursorInZone("forest-zone");

// Zone runtime durumunu al
ZoneRuntimeState state = _zoneSystem.GetState("forest-zone");
// state.Level, state.IsUnlocked, state.YieldPerSecond

// Tüm config listesi (UI için)
IReadOnlyList<ZoneConfigSO> configs = _zoneSystem.Configs;

// Toplam zone geliri (ömür boyu)
long earned = _zoneSystem.TotalZoneEarned;

// Olaylar
ZoneSystem.OnZoneUnlocked += (id) => UnlockZoneUI(id);
ZoneSystem.OnZoneUpgraded += (id, level) => UpdateZoneLevel(id, level);
ZoneSystem.OnZoneEntered += (id) => HighlightZone(id);
ZoneSystem.OnZoneExited += (id) => UnhighlightZone(id);
```

Mod Farkı: PassiveMode vs Aktif Mod

Özellik	PassiveMode = true	PassiveMode = false
Gelir tetikleyici	Her TickEngine tick'i	Cursor zone içindeyken her tick
Hover çarpanı	Her zaman uygulanmaz	Cursor içindeyken uygulanır
Tipik kullanım	Farm, şehir, temel bölgeler	Özel "aktif" bölgeler

Kayıt / Yükleme

```
// Otomatik: RegisterstateProvider ile SaveData.ZoneStates kaydedilir.
// SaveProviderOrder.Zone = 70
```

44. CursorYieldService

Ne yapar: Mouse/parmak hareketini altına çevirir. Üç farklı yield modeli sunar: Hız (ne kadar hızlı hareket edersen o kadar kazanırsın), Mesafe (kaç piksel hareket ettin), Hover (belirli bir noktada bekle ve ısın). GeneratorSystem ve ClickYieldService ile birlikte toplam gelirin parçası olabilir.

Config: CursorActivityConfigSO

Create → Endless Engine → Cursor Activity Config

Alan	Açıklama	Örnek
YieldModel	Speed / Distance / Hover	Speed
BaseYieldPerSecond	Maksimum saniye başına gelir (Speed modunda)	10.0
PixelsPerGold	Kaç piksel = 1 altın (Distance modunda)	50
HoverWarmupSeconds	Hareketsiz beklenecek süre (Hover modunda)	2.0
HoverRadius	Hareketsiz sayılma toleransı (piksel)	5.0
SpeedSmoothTime	Hız smoothing süresi	0.3

Başlatma

```
_cursorYieldService.Initialize(
    config: _cursorActivityConfig,
    economy: _economyService,
    gameFlow: _gameFlowStateMachine
);
```

API

```
// Mevcut gelir hızı (HUD için)
float yps = _cursorYieldService.CurrentYieldPerSecond;

// Toplam cursor geliri (istatistik için)
long earned = _cursorYieldService.TotalCursorEarned;

// Totalleri sıfırla (prestige sonrası)
_cursorYieldService.ResetTotals();
```

Model Karşılaştırması

Model	Ne Ödüllendirir	İdeal Oyun Türü
Speed	Hızlı hareket	Aksiyonlu idle, dash reward
Distance	Toplam mesafe	Lazy kıyma oyunları
Hover	Sabırlı bekleme	Strateji, zen idle

HUD Baęlantısı

```
void Update()
{
    cursorYpsText.text = $"{_cursorYieldService.CurrentYieldPerSecond:F1}/s";
}
```

45. InventoryService

Ne yapar: Slot tabanlı öge envanteri. Her öge türü (`ItemConfigSO`) için stack sayısını tutar; maksimum slot ve maksimum stack boyutu kısıtları uygular. `MergeService` ve `DropResolver` ile entegre çalışır.

Config: ItemConfigSO

Create → Endless Engine → Item Config

Alan	Açıklama	Örnek
ItemId	Benzersiz ID	"iron-ore"
DisplayName	Gösterim adı	"Demir Cevheri"
MaxStackSize	Stack başına maksimum adet	99
Rarity	Common / Rare / Epic / Legendary	Common
Icon	Envanter ikonu	(Sprite)

Başlatma

```
_inventoryService.Initialize(
    allItems: ConfigRegistry.ItemConfigs, // ItemConfigSO[]
    maxSlots: 20                          // 0 = sınırsız
);
_saveService.RegisterStateProvider(_inventoryService);
```

API

```
// Ekle (eklenen miktar döner – stack doluysa daha az olabilir)
int added = _inventoryService.Add("iron-ore", count: 5);

// Çıkar (başarısız olursa false)
bool ok = _inventoryService.Remove("iron-ore", count: 2);

// Miktar sorgula
int count = _inventoryService.GetCount("iron-ore");

// Var mı?
bool has = _inventoryService.Has("iron-ore", count: 3);

// Slot sayısı
int slots = _inventoryService.SlotCount;
int maxSlots = _inventoryService.MaxSlots;

// Tüm stackler (UI için)
IReadOnlyDictionary<string, int> stacks = _inventoryService.Stacks;

// Olaylar
InventoryService.OnInventoryChanged += (itemId, newCount, delta) =>
    inventoryUI.UpdateSlot(itemId, newCount);

InventoryService.OnInventoryFull += (itemId, count) =>
    ShowError($"Envanter dolu! ({count} {itemId} eklenemedi)");
```

MergeService ile Entegrasyon

```
// MergeService Initialize'da InventoryService referansını alır:
_mergeService.Initialize(
    configs: _mergeConfigs,
    inventory: _inventoryService,
    economy: _economyService
);

// Birleştirme – InventoryService otomatik güncellenir
MergeResult result = _mergeService.TryMerge(itemConfig);
```

46. DropResolver

Ne yapar: Ağırlıklı rastgele drop sistemi. Her roll için birden fazla drop üretebilir; entegre pity sayacı belirli sayıda başarısız rolden sonra nadir item garantisi sağlar. Zero-allocation common path.

Config: DropTableConfigSO

Create → Endless Engine → Drop Table Config

Alan	Açıklama	Örnek
TableId	Benzersiz ID	"enemy-drops"
RollsPerUse	Her kullanımda kaç roll atılır	1
PityThreshold	Garantiye kadar roll sayısı	50
PityMinRarity	Garanti edilecek minimum nadirlik	Rare
Entries[]	Her biri Item, Weight, MinCount, MaxCount, Rarity	—

Kullanım

DropResolver bir MonoBehaviour değil — new ile oluşturulur veya Bootstrap'te enjekte edilir.

```
// Oluştur
var dropResolver = new DropResolver();

// Roll at
List<DropResult> drops = dropResolver.Roll(_enemyDropTable);

foreach (var drop in drops)
{
    _inventoryService.Add(drop.Item.ItemId, drop.Count);

    if (drop.WasPityGuaranteed)
        ShowSpecialEffect(drop.Item);
}

// Pity bilgisi (UI için)
int pityProgress = dropResolver.GetPityCounter("enemy-drops");

// Sayacı sıfırla (prestige sonrası)
dropResolver.ResetPityCounter("enemy-drops");
dropResolver.ResetAllPityCounters();
```

DropResult Yapısı

```
// drop.Item          - ItemConfigSO
// drop.Count         - düşen adet
// drop.Rarity        - bu dropin nadirlik seviyesi
// drop.WasPityGuaranteed - pity garantisiyle mi düştü?
```

Düşman Ölümünde Drop Tetikleme

```
HealthSystem.OnEntityDied += (entityId, vfxTag, pos) =>
{
    var drops = _dropResolver.Roll(_enemyDropTable);
    foreach (var d in drops)
        _inventoryService.Add(d.Item.ItemId, d.Count);
};
```

47. PlayerHealthComponent

Ne yapar: Oyuncu can sistemi. `DamageSystem.OnDamageResolved` eventine abone olarak hasarı filtreler ve oyuncuya uygular. I-frame (hasar dokunulmazlığı) penceresi, ölüm gecikmesi ve `IdleRecovery` durum geçişi yönetir. `IPlayerQuery` arayüzünü uygular; `EnemyManager` ve `AutoBattleController` bunu kullanır.

Kullanım

`PlayerHealthComponent` bir `MonoBehaviour`'dur — sahnede `Player GameObject`'ine eklenir.

```
Player/
├─ PlayerInput
├─ InputProviderUnity
└─ PlayerHealthComponent ← buraya
```

Inspector Alanları

Alan	Açıklama	Varsayılan
MaxHP	Maksimum can	100
IFrameDurationSeconds	Hasar sonrası dokunulmazlık süresi	0.5
DeathTransitionDelaySeconds	Ölüm animasyon gecikmesi	1.0
EntityId	<code>DamageSystem</code> için benzersiz ID	Auto (GetInstanceID)

API

```
// Can değerleri (HUD için)
float current = _playerHealth.CurrentHP;
float max     = _playerHealth.MaxHP;

// Durum sorguları
bool invincible = _playerHealth.IsInvincible; // i-frame aktif mi?
bool inIdleRecovery = _playerHealth.IsInIdleRecovery; // ölüm sonrası kurtarma
Vector2 position = _playerHealth.Position; // dünya pozisyonu

// Olaylar
PlayerHealthComponent.OnPlayerHPChanged += (current, max) =>
    hpBar.fillAmount = current / max;

PlayerHealthComponent.OnEntityDied += (id, vfxTag, pos) =>
    PlayDeathAnimation(pos);

PlayerHealthComponent.OnPlayerEnteredIdleRecovery += () =>
    ShowRevivePrompt();
```

IdleRecovery Nedir?

`IdleRecovery`, oyuncunun HP'si sıfıra düştükten `DeathTransitionDelaySeconds` sonra girdiği özel durumdur. Bu sürede `EnemyManager` tüm düşman davranışını duraklatır (`EnemyState.Idle`). Oyun `OnPlayerEnteredIdleRecovery` event'ini ateşler — bu event'e abone olan sistemler (örn. `WaveSpawnManager`) gerekli yeniden başlatma işlemlerini yapar.

DamageDispatchAdapter ile Bağlantı

AutoBattleController 'ın düşman saldırılarını işlemesi için DamageDispatchAdapter MonoBehaviour'una _playerHealth referansı atanmalıdır:

```
DamageDispatchAdapter  
└─ _playerHealth → [PlayerHealthComponent referansı]
```

48. GameFlowState ve RunSummaryData

GameFlowState

Ne yapar: Oyunun üç ana makro durumunu tanımlayan enum. `GameFlowStateMachine` bu geçişleri yönetir; diğer sistemler mevcut durumu sorgular.

```
// Namespace: EndlessEngine.Flow
using EndlessEngine.Flow;

GameFlowState state = _gameFlowStateMachine.CurrentState;

switch (state)
{
    case GameFlowState.Menu:
        // Ana menü, jeneratör ekranı, upgrade ekranı
        // Pasif gelir tick'leri devam eder
        break;

    case GameFlowState.InRun:
        // Aktif koşu – arena aktif, koşu sayacı sayıyor
        // AutoBattleController savaşıyor
        break;

    case GameFlowState.PostRun:
        // Koşu bitti – özet ekranı gösteriliyor
        // Menu'ye dönmeye önce RunSummaryData gösterilir
        break;
}

// Durum değişince
_gameFlowStateMachine.OnStateChanged += (prev, next) =>
{
    if (next == GameFlowState.PostRun)
        ShowRunSummary(_runSessionManager.LastRunSummary);
};
```

RunSummaryData

Ne yapar: Tek bir koşunun sonuçlarını tutan değişmez snapshot. `RunSessionManager` koşu bitince oluşturur; `PostRunScreen` 'e aktarılır.

```
// Namespace: EndlessEngine.Statistics
using EndlessEngine.Statistics;

// RunSummaryData alanları:
RunSummaryData summary = _runSessionManager.LastRunSummary;

Debug.Log($"Süre: {summary.DurationSeconds:F0}s");
Debug.Log($"Altın: {summary.GoldEarned}");
Debug.Log($"Öldürme: {summary.KillCount}");
Debug.Log($"En Yüksek Dalga: {summary.MaxWave}");
Debug.Log($"Prestige mi? {summary.PrestigePerformed}");
Debug.Log($"Cascade: {summary.CascadeMultiplier:F2}x");
Debug.Log($"Son Gelir: {summary.FinalIncomeRate:F1}/s");
```

RunSummaryData Factory

```
// Kendi koşu özeti oluşturmak için (özel RunSessionManager'lar):
var summary = RunSummaryData.Create(
    startTime:      DateTime.UtcNow - TimeSpan.FromSeconds(duration),
    endTime:        DateTime.UtcNow,
    goldEarned:      totalGold,
    killCount:       kills,
    maxWave:         wave,
    prestigeCountAtStart: startPrestige,
    prestigePerformed: didPrestige,
    upgradesAccepted: upgradeCount,
    cascadeMultiplier: cascade,
    finalIncomeRate:  incomeRate
);
```

PostRun UI Entegrasyonu

```
void ShowRunSummary(RunSummaryData s)
{
    durationText.text = $"{s.DurationSeconds:F0}s";
    goldText.text     = FormatGold(s.GoldEarned);
    killText.text     = s.KillCount.ToString();
    waveText.text     = $"Dalga {s.MaxWave}";
    cascadeText.text  = $"×{s.CascadeMultiplier:F2}";
    incomeText.text   = $"${s.FinalIncomeRate:F1}/s";

    if (s.PrestigePerformed)
        prestigeBadge.SetActive(true);
}
```

49. IdleModule

Ne yapar: Tüm Endless Engine modüllerinin uygulaması gereken yaşam döngüsü arayüzü. `ModuleRegistry` bu arayüzü kullanan modüllerin başlatma sırasını, tick aboneliğini ve kapatma işlemini yönetir.

Arayüz

```
// Namespace: EndlessEngine.Modules
public interface IIdleModule
{
    string ModuleId { get; } // "economy", "skill-tree", "research" vb.
    string[] Dependencies { get; } // Önce başlatılması gereken modüllerin ID'leri
    int InitOrder { get; } // Aynı tier içinde sıralama (küçük = önce)
    bool ReceivesTick { get; } // false ise tick çağrılmaz (veri modülleri için)

    IEnumerator Init(); // Coroutine – senkronsa sadece yield return null
    void Tick(float dt); // TickEngine.OnTick bağlantısı
    void Shutdown(); // Olay aboneliklerini temizle
}
```

Kendi Modülünüzü Yazma

```
public class MyProductionModule : MonoBehaviour, IIdleModule
{
    public string ModuleId => "my-production";
    public string[] Dependencies => new[] { "economy", "tick-engine" };
    public int InitOrder => 100;
    public bool ReceivesTick => true;

    private EconomyService _economy;

    public IEnumerator Init()
    {
        _economy = FindObjectOfType<EconomyService>();
        yield return null; // senkron init – coroutine gerektirmez
    }

    public void Tick(float dt)
    {
        _economy.AddResources(5.0 * dt);
    }

    public void Shutdown()
    {
        // Event aboneliklerini temizle
    }
}
```

ModuleRegistry'ye Kaydetme

```
_moduleRegistry.Register(GetComponent<MyProductionModule>());
// Tüm modüller kaydedildikten sonra:
yield return StartCoroutine(_moduleRegistry.InitializeAll());
```

Dikkat: Döngüsel Bağımlılık

Modül `Dependencies` dizisinde döngüsel bağımlılık belirtilirse `ModuleRegistry` başlatma sırasında exception fırlatır. `Init()` içinde bağımlılıkları elle çözmek yerine her zaman `Dependencies` kullanın.

BÖLÜM I

Araçlar ve Testler

50. Editor Araçları

51. Test Stratejisi

50. Editor Araçları

Tüm araçlar: **Tools** → **Endless Engine** → ...

Araç	Ne Yapar	Kısa Kullanım
New Game Wizard	Oyun tipine göre tüm temel config'leri otomatik oluşturur	Oyun tipi seç → Create
Content Pack Wizard	Yeni Realm için 9 SO + RealmPackSO + Registry kaydı	Realm adı gir → Create All
Upgrade Tree Editor	DAG upgrade ağacını görsel düzenleme	Düğüm sürükleyip, ok çiz
Skill Tree Editor	Skill ağacını görsel düzenleme	Aynı şekilde
Trait Tree Editor	Trait ağacını görsel düzenleme	Aynı şekilde
Generator Editor	Generator asset'lerini tablo şeklinde toplu düzenleme	Açık → Düzenle
Generator Asset Creator	Birden fazla GeneratorConfigSO'yu toplu oluşturur	İsimleri gir → Batch Create
ID Registry Window	Tüm SO ID'leri tarar; duplikat ve yetim ID'leri listeler	Scan → Jump ile git
Config Validator	Tüm config SO'ları doğrulama kurallarına göre tarar	Validate Configs
Economy Simulator	Gelir eğrisini simüle eder	Parametreleri gir → Simulate
Economy Tuning Window	Config değerlerini oyun açıkken canlı düzenle	Editor mod only
Schema Bump Utility	SchemaVersion artırır, migrasyon şablonu oluşturur	Bump Version

New Game Wizard Oyun Tipleri

Tip	Oluşturulan Sistemler
Pure Idle	Jeneratör + pasif gelir
Clicker Idle	Tıklama + jeneratör
Merge Idle	Merge sistemi + ekonomi
Prestige-Heavy	Tüm prestige katmanları
Wave Idle	AutoBattle + wave + upgrade
Incremental RPG	Tüm sistemler dahil

51. Test Stratejisi

Katman 1 — Unit Tests

Tek sistem, izole, bağımlılık yok. NUnit `[Test]`, `EditMode`.

```
[Test]
public void AddResources_BelowCap_IncreasesBalance()
{
    var svc = new GameObject().AddComponent<EconomyService>();
    svc.InjectStateForTesting(currentResources: 100, hardCap: 1000, startingGold: 0);
    svc.AddResources(50);
    Assert.AreEqual(150, svc.CurrentResources, 0.01);
}
```

Katman 2 — Integration Tests

Birden fazla sistem birlikte. `Assets/Tests/integration/` altında.

```
[Test]
public void OnBeforeSave_DestroyedTarget_WritesRespawnEntry()
{
    _target.ApplyDamage(_targetConfig.MaxHP); // hedefi yok et
    var save = new SaveData();
    save.EnsureDefaults();
    _service.OnBeforeSave(save);

    Assert.AreEqual(1, save.ClickLoopState.TargetStates.Count);
    Assert.IsTrue(save.ClickLoopState.TargetStates.First().Value.IsRespawning);
}
```

Katman 3 — Performance Tests

Kritik yollarda GC allocation ve frame süresi.

```
[Test, Performance]
public void AddResources_HotPath_ZeroAllocation()
{
    Measure.Method(() => _economy.AddResources(1.0))
        .GC()
        .Run();
}
```

Test Çalıştırma

Window → General → Test Runner → EditMode → Run All

Önemli Test Kalıpları

ConfigRegistry her testte sıfırlanmalı:

```
[SetUp] public void Setup() => ConfigRegistry.InjectForTesting(economy: _cfg);
[TearDown] public void Teardown() => ConfigRegistry.ClearForTesting();
```

Input System testleri için:

```
Press(_keyboard.wKey);
InputSystem.Update(); // EditMode'da manuel güncelleme şart
Assert.AreEqual(1f, _provider.GetMoveVector().y, 0.05f);
```

MonoBehaviour testlerinde config enjeksiyonu:

```
_go = new GameObject();
_go.SetActive(false); // Awake'i ertele
_go.AddComponent<BoxCollider2D>();
_target = _go.AddComponent<ClickTarget>();
SetField(_target, "_config", _config); // reflection ile enjekte et
_go.SetActive(true); // şimdi Awake çalışır

static void SetField(object t, string n, object v)
{
    var f = t.GetType().GetField(n,
        BindingFlags.NonPublic | BindingFlags.Instance);
    f?.SetValue(t, v);
}
```

BÖLÜM J

Oyun Yapım Tarifleri

52. Tarif 1: Klasik Idle

53. Tarif 2: Aktif Clicker

54. Tarif 3: Hasat Loop

55. Tarif 4: Idle-vs

56. Tarif 5: Merge Idle

57. Tarif 6: Prestige-Heavy RPG Idle

52. Tarif 1: Klasik Idle

Hedef: Cookie Clicker benzeri — tıkla, jeneratör al, yükseltme yap.

Kullanılan Sistemler

EconomyService · TickEngine · GeneratorSystem · PassiveIncomeService · ClickYieldService · UpgradeTreeService · SaveService · OfflineTimeCalculator

Kurulum

Tools → New Game Wizard → Pure Idle → Create

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  GeneratorSystem
  PassiveIncomeService
  ClickYieldService
  UpgradeTreeService
  OfflineTimeCalculator
```

Bootstrap Script'i (Adım Açıklamalarıyla)

```
private async void Awake()
{
    // ADIM 1: Config'leri yükle.
    // Addressables akışında ConfigLoadingService bunu yapar (otomatik).
    // Test/direkt sahnede InjectForTesting kullanılır.
    ConfigRegistry.InjectForTesting(economy: _economyConfig);

    // ADIM 2: UpgradeTree önce başlatılır çünkü EconomyService ona bağımlıdır.
    // HandleConfigsLoaded() → ConfigRegistry.Upgrades'i okur ve DAG'ı kurar.
    _upgradeTree.HandleConfigsLoaded();

    // ADIM 3: EconomyService – UpgradeTree'den sonra, diğer her şeyden önce.
    // Neden? GeneratorSystem ve PassiveIncome, EconomyService.AddResources()'a çağrı yapar.
    _economyService.Initialize(_upgradeTree, _saveService);

    // ADIM 4: GeneratorSystem – EconomyService'e bağımlı.
    _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService, _saveService);

    // ADIM 5: PassiveIncomeService – GeneratorSystem ve EconomyService'e bağımlı.
    // null = GameFlowStateMachine yok (Pure Idle'da run/menu ayrımı yok)
    _passiveIncome.Initialize(_generatorSystem, _economyService, null);

    // ADIM 6: ClickYieldService – sadece EconomyService'e bağımlı.
    _clickYield.Initialize(
        config: _clickSourceConfig,
        economy: _economyService,
        passiveYieldGetter: null // YieldRateClickFraction kullanmak istiyorsanız doldurun
    );
    _clickYield.SetInputProvider(_inputProvider);

    // ADIM 7: OfflineTimeCalculator için HİÇBİR ŞEY ÇAĞIRILMAZ.
    // Awake()'te SaveService.OnSaveLoaded'a zaten abone olmuştur.

    // ADIM 8: SaveService'e sağlayıcıları kaydet – LoadAsync()'den ÖNCE olmalı.
```

```
// Neden önce? LoadAsync → OnSaveLoaded → OnAfterLoad(saveData) sırasıyla çalışır.
// Sağlayıcı kayıtlı değilse OnAfterLoad çağrılmaz, kayıt yüklenmez.
_saveService.RegisterServiceProvider(_economyService); // Order: 10
_saveService.RegisterServiceProvider(_upgradeTree); // Order: 20
_saveService.RegisterServiceProvider(_generatorSystem); // Order: 50

// ADIM 9: Kayıtları yükle – her şey hazır olduktan sonra.
// Bu çağrı: disk okuma → SaveData oluştur → her sağlayıcının OnAfterLoad'ı
// → OnSaveLoaded event'i → OfflineTimeCalculator hesabı
await _saveService.LoadAsync();
}
```

Tıklama Butonu

```
// UI butonunun OnClick event'ine bağlayın:
public void OnClickGold() => _clickYield.ProcessClick();
```

Jeneratör UI

```
void RefreshUI()
{
    foreach (var cfg in ConfigRegistry.Generators)
    {
        var btn = GetButtonFor(cfg.GeneratorId);
        btn.countText.text = _generatorSystem.GetCount(cfg.GeneratorId).ToString();
        btn.costText.text = FormatGold(_generatorSystem.GetNextCost(cfg.GeneratorId));
        btn.ypsText.text = $"{_generatorSystem.CalculateTotalYield():F1}/s";
    }
}
```

Offline Popup

```
OfflineTimeCalculator.OnOfflineGainCalculated += (gold, secs) =>
{
    if (gold > 0)
        offlinePopup.Show($"{FormatGold(gold)} kazandınız ({FormatTime(secs)} çevrimdışıydınız)");
};
```

53. Tarif 2: Aktif Clicker

Hedef: Ekranaya yerleştirilen hedef nesnelere tıkla, yok et, altın kazan, yeniden doğmasını bekle.

Kullanılan Sistemler

EconomyService · TickEngine · ClickLoopService · ClickTarget · ClickTargetSpawner · ClickLoopOfflineCalculator · ClickLoopHUDController · UpgradeTreeService · StatisticsService · SaveService

Kurulum

Tools → New Game Wizard → Clicker Idle → Create

Sahne Yapısı

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  UpgradeTreeService
  StatisticsService
  ClickLoopService
  ClickLoopOfflineCalculator
  VFXController
Player/
  PlayerInput
  InputProviderUnity
World/
  CoinBag_1 [ClickTarget + BoxCollider2D]
  CoinBag_2 [ClickTarget + BoxCollider2D]
  CoinBag_3 [ClickTarget + BoxCollider2D]
UI/
  ClickLoopHUD [ClickLoopHUDController]
```

Config'leri Oluşturun

Create → Endless Engine → Click Loop → Click Loop Config (ComboDecayDelay=1.5, MaxComboMultiplier=8)
Create → Endless Engine → Click Loop → Click Target Config (TargetId="coin-bag", MaxHP=5, BaseYield=30)

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);

    _upgradeTree.HandleConfigsLoaded();
    _economyService.Initialize(_upgradeTree, _saveService);
    _statisticsService.Initialize(ConfigRegistry.StatDefinitions);

    _clickLoopService.Initialize(
        config: _clickLoopConfig,
        economy: _economyService,
        input: _inputProvider,
        targetLayer: LayerMask.GetMask("ClickableTargets"),
        statistics: _statisticsService,
        vfx: _vfxController
    );

    _clickLoopHUD.Initialize(_clickLoopService, _clickLoopConfig);

    _clickLoopOfflineCalc.Initialize(
        config: _clickLoopConfig,
        economy: _economyService,
        targetConfigs: _clickTargetConfigs // ClickTargetConfigS0[] - tüm hedef türleri
    );
    _saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_statisticsService);
    _saveService.RegisterStateProvider(_clickLoopService);

    await _saveService.LoadAsync();
}

private void OnDestroy()
{
    _saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
}
```

ClickTarget Sahnede Kurulumu

Her hedef GameObject için:

1. ClickTarget component ekle.
2. Inspector'da _config alanına ClickTargetConfigS0 asset'ini ata.
3. Layer'ı ClickableTargets olarak ayarla.

Upgrade Yükseltmeleri

```
Create → Upgrade Node Config
NodeId = "click-dmg-1"
DisplayName = "Tıklama Hasarı I"
AffectedStat = ClickDamage
EffectPerRank = 0.2 (+%20 her rank)
MaxRank = 5
BaseCost = 50
```

```
// Goldı göster
EconomyService.OnResourcesChanged += (current, _) =>
    goldText.text = FormatGold(current);

// Combo göster
_clickLoopService.OnComboChanged += (mult) =>
    comboText.text = $"x{mult:F1}";

// Kritik flaş
_clickLoopService.OnCrit += (_) =>
    StartCoroutine(CritFlashCoroutine());
```

54. Tarif 3: Hasat Loop

Hedef: Mouse'u/parmağı dünya nesneleri üzerinde sürükleyerek hasar ver, altın kazan.

Kullanılan Sistemler

EconomyService · TickEngine · HarvestLoopService · HarvestCursor · HarvestNode · HarvestNodeSpawner · HarvestOfflineCalculator · HarvestHUDController · UpgradeTreeService · StatisticsService · VFXController · SaveService

Sahne Yapısı

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  UpgradeTreeService
  StatisticsService
  HarvestLoopService
  HarvestOfflineCalculator
  VFXController
World/
  HarvestCursor [HarvestCursor + (ayrı G0)]
  Ore_1 [HarvestNode + CircleCollider2D]
  Ore_2 [HarvestNode + CircleCollider2D]
  Ore_3 [HarvestNode + CircleCollider2D]
Player/
  PlayerInput
  InputProviderUnity
UI/
  HarvestHUD [HarvestHUDController]
```

Config'leri Oluşturun

```
Create → Endless Engine → Harvest → Harvest Area Config
(BaseRadius=1.5, BaseTickInterval=0.25, ComboDecayDelay=1.5)

Create → Endless Engine → Harvest → Harvest Node Config
(NodeId="gold-ore", MaxHP=20, DamagePerTick=2, BaseYield=100, RespawnSeconds=10)
```

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);

    _upgradeTree.HandleConfigsLoaded();
    _economyService.Initialize(_upgradeTree, _saveService);
    _statisticsService.Initialize(ConfigRegistry.StatDefinitions);

    _harvestCursor.Inject(_inputProvider);

    _harvestLoopService.Initialize(
        cursor: _harvestCursor,
        config: _harvestAreaConfig,
        economy: _economyService,
        statistics: _statisticsService,
        vfx: _vfxController
    );

    _harvestHUD.Initialize(_harvestLoopService, _harvestCursor, _harvestAreaConfig);

    _harvestOfflineCalc.Initialize(
        config: _harvestAreaConfig,
        economy: _economyService,
        nodeConfigs: _harvestNodeConfigs // HarvestNodeConfigS0[] - tüm node türleri
    );
    _saveService.OnSaveLoaded += _harvestOfflineCalc.HandleSaveLoaded;

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_statisticsService);
    _saveService.RegisterStateProvider(_harvestLoopService);

    await _saveService.LoadAsync();
}

private void OnDestroy()
{
    _saveService.OnSaveLoaded -= _harvestOfflineCalc.HandleSaveLoaded;
}
```

HarvestNode Sahnede Kurulumu

Her node GameObject için:

1. `HarvestNode` component ekle.
2. Inspector'da `_config` alanına `HarvestNodeConfigS0` asset'ini ata.
3. Layer'ı `HarvestNodes` olarak ayarla.

Upgrade Yükseltmeleri

```
Create → Upgrade Node Config
NodeId = "harvest-radius-1"
DisplayName = "Hasat Yarıçapı I"
AffectedStat = HarvestRadius
EffectPerRank = 0.15 (+%15 yarıçap artışı)
MaxRank = 5
```

55. Tarif 4: Idle-vs

Hedef: Otomatik savaş + dalga sistemi + upgrade kartı + prestige.

Kullanılan Sistemler

Tarif 1'deki her şey + WaveSpawnManager · AutoBattleController · DamageSystem · EnemyManager · HealthSystem · UpgradeApplicationSystem · PrestigeStateManager · InputProviderUnity

Ek Config'ler

```
Create → Wave Config (TotalWavesPerRun=30, WaveTransitionDelaySeconds=2)
Create → Player Config (BaseAttackDamage=10, BaseMaxHP=100, BaseCritChance=0.05)
Create → Prestige Config (MinWaveToPrestige=10, BaseMultiplierPerPrestige=0.1)
```

Tam Bootstrap Script'i (Adım Açıklamalarıyla)

```
private async void Awake()
{
    // ADIM 1: Config'leri yükle.
    // Prodüksiyonda ConfigLoadingService bunu otomatik yapar.
    // Burada prestige ve wave config'leri de gerekli – PrestigeStateManager ve
    // WaveSpawnManager bunları ConfigRegistry'den otomatik okur.
    ConfigRegistry.InjectForTesting(
        economy: _economyConfig,
        wave: _waveConfig,
        player: _playerConfig,
        prestige: _prestigeConfig,
        schema: _schemaVersion
    );

    // ADIM 2: UpgradeTree önce başlatılır – EconomyService'e bağımlı.
    _upgradeTree.HandleConfigsLoaded();

    // ADIM 3: EconomyService
    _economyService.Initialize(_upgradeTree, _saveService);

    // ADIM 4: GeneratorSystem
    _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService, _saveService);

    // ADIM 5: PassiveIncome
    _passiveIncome.Initialize(_generatorSystem, _economyService, _gameFlowStateMachine);

    // ADIM 6: WaveSpawnManager
    // WaveConfigSO Initialize'a GEÇİRİLMEZ – Awake()'de ConfigRegistry.Wave'den okunur.
    _waveManager.Initialize(
        enemyManager: _enemyManager,
        saveNotifier: _saveService,
        healthSystem: _healthSystem // opsiyonel – oyuncu ölümünde wave durdurma
    );
}
```

```

// ADIM 7: AutoBattleController
// IUpgradeStatProvider oluştur - doğrudan UpgradeApplicationSystem static referansı
// kullanılmaz; BaseStatUpgradeProvider arayüzü sağlar.
var statProvider = new BaseStatUpgradeProvider(ConfigRegistry.Player);
_abc.Initialize(
    enemyManager: _enemyManager,
    waveSpawnManager: _waveManager,
    statProvider: statProvider,
    playerConfig: ConfigRegistry.Player,
    waveConfig: ConfigRegistry.Wave,
    playerId: _playerHealthComponent.gameObject.GetInstanceID()
);
_abc.SetPlayerQuery(_playerHealthComponent); // IPlayerQuery - PlayerHealthComponent uygular

// ADIM 8: PrestigeStateManager KURULUMU
// Initialize() YOKTUR - ConfigRegistry.Prestige'i OnEnable'da okur.
// Tek yapılacak: WaveSpawnManager köprüsü + SaveService kaydı.
WaveSpawnManager.OnWaveStarted += (wave) => _prestigeManager.SetCurrentWave(wave);

// Prestige akışında run efektlerini temizle:
PrestigeStateManager.OnPrestigeStarted += () =>
    UpgradeApplicationSystem.ClearRunEffects();

// ADIM 9: SaveService sağlayıcılarını kaydet - LoadAsync'ten ÖNCE
_saveService.RegisterStateProvider(_economyService); // Order: 10
_saveService.RegisterStateProvider(_upgradeTree); // Order: 20
_saveService.RegisterStateProvider(_prestigeManager); // Order: 30
_saveService.RegisterStateProvider(_generatorSystem); // Order: 50
_saveService.RegisterStateProvider(_waveManager); // Order: 40

// ADIM 10: Kayıtları yükle
await _saveService.LoadAsync();

// ADIM 11: Savaşı başlat - LoadAsync SONRASINDA çağrılmalı.

// Neden? Kayıt yüklenince dalga numarası restore edilir;
// StartFirstWave() kayıttan gelen numaradan devam eder.
_waveManager.StartFirstWave();
_abc.StartCombat();
}

```

Dalga Sonu Upgrade Ekranı

```

WaveSpawnManager.OnWaveComplete += (wave) =>
{
    if (wave % ConfigRegistry.Wave.UpgradeSelectionWaveInterval == 0)
    {
        _abc.StopCombat();
        var availableNodes = _upgradeTree.GetAvailableNodes();

        // 3 kart rastgele seç (SelectionWeight'e göre ağırlıklı tercih tercih edilebilir)
        var cards = availableNodes
            .OrderBy(_ => UnityEngine.Random.value)
            .Take(3)
            .Select(n => n.Config)
            .ToArray();

        upgradeScreen.Show(cards, (chosen) =>
        {
            _economyService.TryPurchase(chosen.NodeId);
            upgradeScreen.Hide();
            _abc.NotifyUpgradeSelected(); // savaşı yeniden başlatır
        });
    }
};

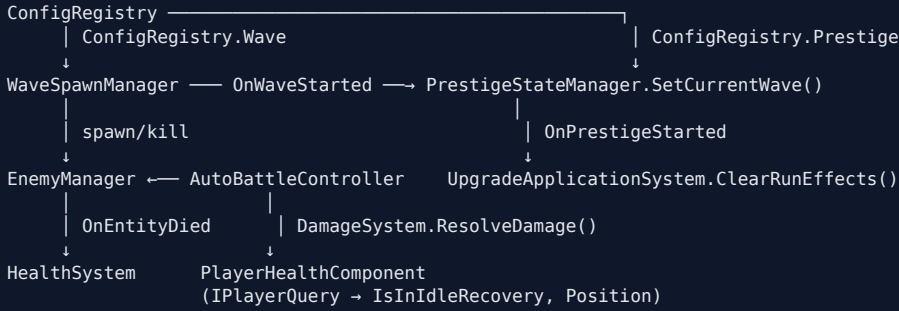
```

Prestige Butonu

```
void Update()
{
    // CanPrestige → MinWaveToPrestige koşulu + SetCurrentWave bağlantısı şart
    prestigeBtn.interactable = _prestigeManager.CanPrestige;
    multiplierText.text      = $"x{_prestigeManager.GetPermanentMultiplier():F2}";
}

public void OnPrestigeClick()
{
    if (_prestigeManager.TryPrestige())
    {
        // OnPrestigeStarted → EconomyService, WaveManager, UpgradeApplicationSystem sıfırlanır
        // OnPrestigeComplete → yeni çarpan gösterilir
        // WaveManager.ResetForNewRun() prestige akışı içinde zaten çağrılır
        _waveManager.StartFirstWave(); // Prestige sonrası yeni run başlat
        _abc.StartCombat();
    }
}
```

Tüm Sistem Bağlantısı Özeti



56. Tarif 5: Merge Idle

Hedef: Merge board + pasif gelir + ekonomi.

Kullanılan Sistemler

EconomyService · MergeService · GeneratorSystem · PassiveIncomeService · UpgradeTreeService · SaveService

Kilit Adımlar

```
// İki objeyi birleştir
public void OnItemDropped(MergeItem a, MergeItem b, Vector2Int pos)
{
    if (a.Tier != b.Tier) return;

    var result = _mergeService.TryMerge(a, b);
    if (!result.Success) return;

    gridView.Remove(a.Position);
    gridView.Remove(b.Position);
    gridView.Place(result.ResultItem, pos);
}

// Yüksek tier merge → anlık altın bonusu
MergeService.OnMergeCompleted += (r) =>
    _economyService.AddResources(r.ResultItem.Tier * 10.0);
```

57. Tarif 6: Prestige-Heavy RPG Idle

Hedef: Derin prestige + ascension katmanları + skill tree + trait sistemi + araştırma.

Kullanılan Sistemler

Tarif 4'teki her şey + AscensionStateManager · SkillTreeService · TraitService · ResearchService · MilestoneTracker · StatisticsService

Ascension Kurulumu

```
Create → Ascension Layer Config
LayerIndex = 1
RequiredPrestigeCount = 10
PermanentMultiplierPerAscension = 2.0
```

```
_ascensionManager.Initialize(
    database: _ascensionDatabase,
    prestigeManager: _prestigeManager,
    saveService: _saveService,
    economyService: _economyService
);
_saveService.RegisterStateProvider(_ascensionManager);

AscensionStateManager.OnAscensionComplete += (layer, count, cascade) =>
    Debug.Log($"Ascension L{layer} #{count} | Cascade: {cascade:F2}x");
```

Prestige → Skill Puanı

```
PrestigeStateManager.OnPrestigeComplete += (count, mult) =>
    _skillTree.AddPoints(1);
```

Ascension → Trait Seçimi

```
AscensionStateManager.OnAscensionStarted += (layer) =>
{
    // HasPendingSelection: TraitService seçim beklediğinde true
    if (_traitService.HasPendingSelection)
    {
        // Burada mevcut trait listesini göstermek için traitService.ChosenTraitIds
        // kullanılır; kullanıcıya sunulan seçenekler OnTraitSelectionAvailable event'iyle gelir
    }
};

TraitService.OnTraitSelectionAvailable += (options) =>
    traitScreen.Show(options, chosen => _traitService.ChooseTrait(chosen.TraitId));
```

Araştırma Kuyruğu

```
_researchService.Initialize(ConfigRegistry.ResearchTrees, _economyService, null);
_tickEngine.OnTick += _researchService.OnTick;
_saveService.RegisterServiceProvider(_researchService);

public void QueueResearch(string treeId, string nodeId)
{
    if (_researchService.EnqueueNode(treeId, nodeId))
        researchUI.RefreshQueue();
}

ResearchService.OnNodeCompleted += (tree, node) =>
    ShowToast($"Araştırma tamamlandı: {node}");
```

BÖLÜM K

Sorun Giderme

58. Sorun Giderme

58. Sorun Giderme

"ConfigNotLoadedException" veya null ref config hatası

Config'e `OnConfigsLoaded` tetiklenmeden önce erişiyorsunuz.

```
// Çözüm: Sisteminizi bu event'e abone edin
ConfigRegistry.OnConfigsLoaded += () => { Initialize(); };
```

Testlerde:

```
[SetUp]    public void Setup()    => ConfigRegistry.InjectForTesting(economy: _cfg);
[TearDown] public void Teardown() => ConfigRegistry.ClearForTesting();
```

Testlerde Input System her zaman 0 döndürüyor

EditMode [UnityTest] 'te `yield return null` `InputSystem.Update()`'i tetiklemez.

```
// Çözüm: [Test] kullanın ve Press() sonrası manuel güncelleme yapın
[Test]
public void WKey_ReturnsUpVector()
{
    Press(_keyboard.wKey);
    InputSystem.Update(); // şart
    Assert.AreEqual(1f, _provider.GetMoveVector().y, 0.05f);
}
```

ClickTarget / HarvestNode Awake'te config null

Test ortamında `SetActive(true)` çağrısından önce config enjekte edilmemiş.

```
// Çözüm: Önce devre dışı oluşturun, config'i enjekte et, sonra aktif et
_go = new GameObject();
_go.SetActive(false);
_go.AddComponent<BoxCollider2D>();
_target = _go.AddComponent<ClickTarget>();
SetField(_target, "_config", _config); // reflection
_go.SetActive(true);                  // şimdi Awake çalışır
```

ClickLoopService ve ClickYieldService aynı anda altın veriyor

Bootstrap otomatik olarak uyarır ve `ClickYieldService`'i devre dışı bırakır. Bunu kasıtlı olarak tetiklemek için:

- `ClickYieldService` kullanıyorsanız `_clickLoopService` alanını Inspector'da **boş bırakın**.
- `ClickLoopService` kullanıyorsanız `_clickYieldService` alanını **boş bırakın**.

Prestige sonrası stat güncellenmiyor

`PrestigeStateManager.Initialize()` metodu yoktur — çağırmayın. Sorun başka bir yerde olabilir:

```
// 1. ConfigRegistry'ye prestige config eklenmiş mi kontrol edin:
ConfigRegistry.InjectForTesting(prestige: _prestigeConfig);

// 2. WaveSpawnManager ile dalga numarası iletişimi kurulmuş mu?
WaveSpawnManager.OnWaveStarted += (wave) => _prestigeManager.SetCurrentWave(wave);

// 3. UpgradeApplicationSystem.ClearRunEffects() prestige akışında çağrılıyor mu?
PrestigeStateManager.OnPrestigeStarted += () => UpgradeApplicationSystem.ClearRunEffects();

// 4. SaveService'e RegisterStateProvider çağrıldı mı?
_saveService.RegisterStateProvider(_prestigeManager);
```

Çevrimdışı hesap çalışmıyor

SaveService.OnSaveLoaded olayına abone olunduğundan ve HandleSaveLoaded imzasının (SaveData data, bool isNewGame) olduğundan emin olun:

```
_saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;
// OnDestroy'da:
_saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
```

BigDouble kayıt/yükleme uyumsuzluğu

EconomyConfigS0.NumberBackend değeri değiştirildiyse eski kayıtlar uyumsuz olur.

```
Tools → Endless Engine → Schema Bump Utility → Bump Version
// Oluşturulan MigrationV{N}.cs dosyasında NumberBackendName kontrolü ekleyin
```

HarvestNode veya ClickTarget kayıt sonrası senkronize değil

SaveService.RegisterStateProvider() Bootstrap'te çağrıldığından ve sıranın SaveConstants.SaveProviderOrder.Harvest (88) / SaveConstants.SaveProviderOrder.ClickLoop (89) olduğundan emin olun.